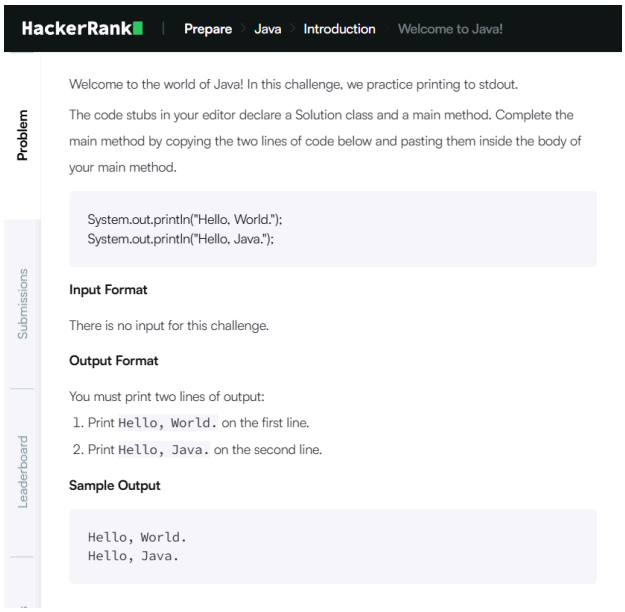
HackerRank Java Programs

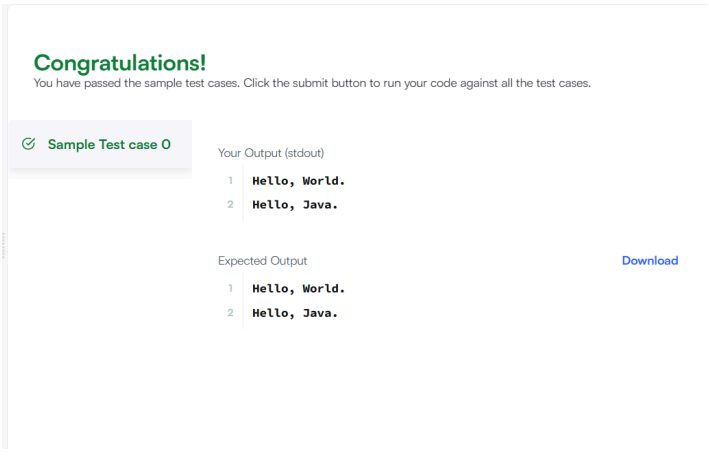
1. Question :



Code :

```
public class Solution {

    public static void main(String[] args) {
        System.out.println("Hello, World.");
        System.out.println("Hello, Java.");
    }
}
```

Output :



2. Question :

Task

In this challenge, you must read **3** integers from stdin and then print them to stdout. Each integer must be printed on a new line. To make the problem a little easier, a portion of the code is provided for you in the editor below.

Input Format

There are **3** lines of input, and each line contains a single integer.

Sample Input

```
42
100
125
```

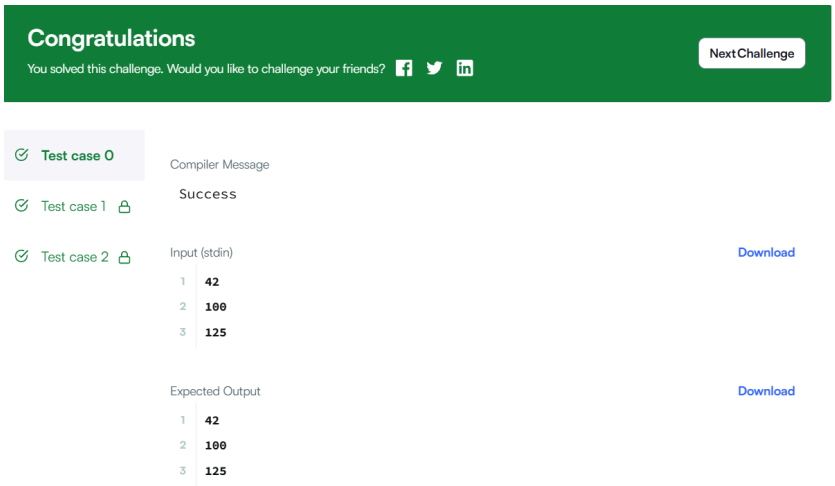
Sample Output

```
42
100
125
```

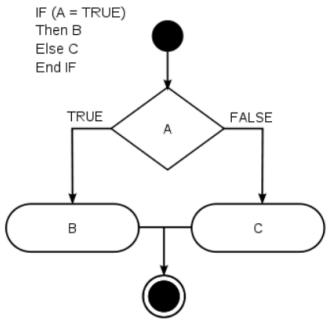
Code :

```
import java.util.*;

public class Solution {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int a = scan.nextInt();
        int b = scan.nextInt();
        int c = scan.nextInt();

        System.out.println(a);
        System.out.println(b);
        System.out.println(c);
    }

}
```

Output :



3 . Question :



Source: Wikipedia

Task

Given an integer, n, perform the following conditional actions:

- If n is odd, print `Weird`
- If n is even and in the inclusive range of 2 to 5, print `Not Weird`
- If n is even and in the inclusive range of 6 to 20, print `Weird`
- If n is even and greater than 20, print `Not Weird`

Complete the stub code provided in your editor to print whether or not n is weird.

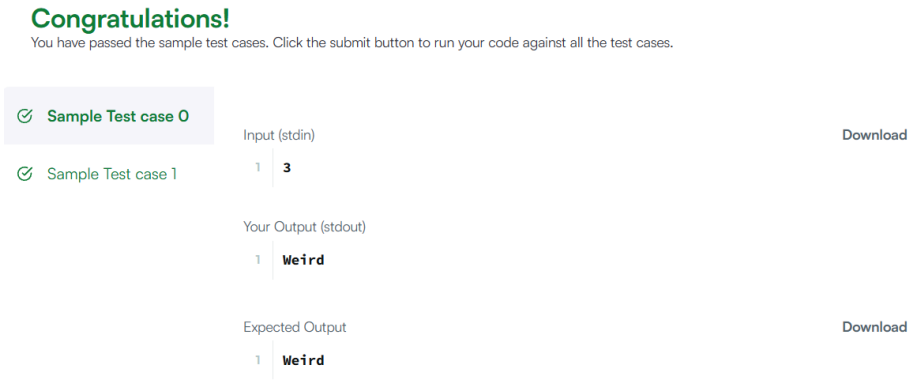
Input Format

A single line containing a positive integer, n.

Code :

```java
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.regex.*;
import java.util.Scanner;

public class Solution {



    private static final Scanner scanner = new Scanner(System.in);

    public static void main(String[] args) {
        int N = scanner.nextInt();
        scanner.skip("(\r\n|[\n\r\u2028\u2029\u0085])?");
        if (N%2!=0){
            System.out.println("Weird");
        }else if (N%2==0 && N>1 && N<6){
            System.out.println("Not Weird");
        }else if (N%2==0 && N>5 && N<21){
            System.out.println("Weird");
        }else if (N%2==0 && N>20){
            System.out.println("Not Weird");
        }
```

Output :



4 . Question :

In this challenge, you must read an integer, a double, and a String from stdin, then print the values according to the instructions in the Output Format section below. To make the problem a little easier, a portion of the code is provided for you in the editor.

Note: We recommend completing Java Stdin and Stdout I before attempting this challenge.

Input Format

There are three lines of input:

1. The first line contains an integer.
2. The second line contains a double.
3. The third line contains a String.

Output Format

There are three lines of output:

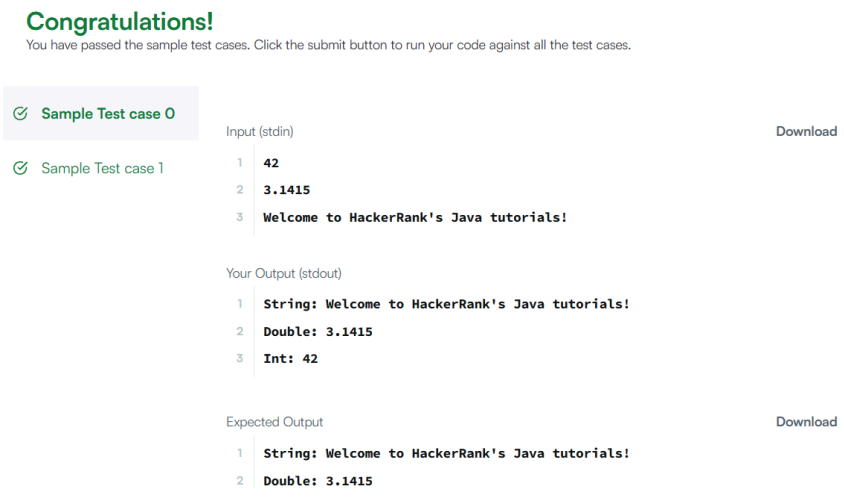
1. On the first line, print `String:` followed by the unaltered String read from stdin.
2. On the second line, print `Double:` followed by the unaltered double read from stdin.
3. On the third line, print `Int:` followed by the unaltered integer read from stdin.

To make the problem easier, a portion of the code is already provided in the editor.

Code :

```java
import java.util.Scanner;
import java.lang.String;

public class Solution {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int i = scan.nextInt();
        double d = scan.nextDouble();
        scan.nextLine();
        String s = scan.nextLine();


        System.out.println("String: " + s);
        System.out.println("Double: " + d);
        System.out.println("Int: " + i);
    }
}
```

Output :



5 . Question :

Java's System.out.printf function can be used to print formatted output. The purpose of this exercise is to test your understanding of formatting output using printf.

To get you started, a portion of the solution is provided for you in the editor; you must format and print the input to complete the solution.

Input Format

Every line of input will contain a String followed by an integer.

Each String will have a maximum of 10 alphabetic characters, and each integer will be in the inclusive range from 0 to 999.

Output Format

In each line of output there should be two columns:

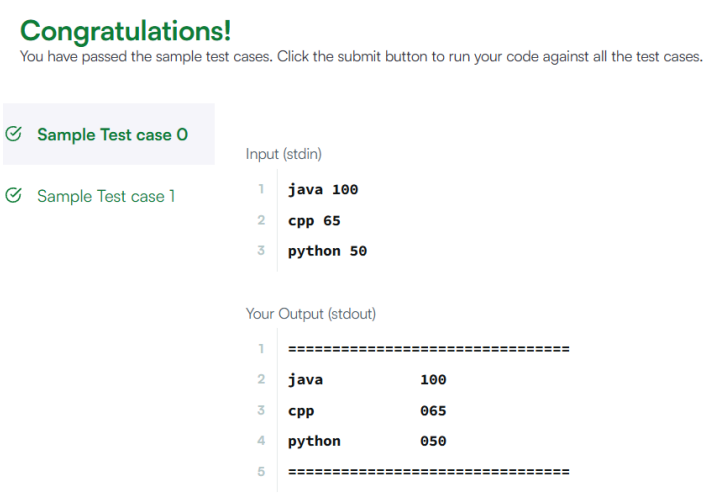
The first column contains the String and is left justified using exactly 15 characters.

The second column contains the integer, expressed in exactly 3 digits; if the original input has less than three digits, you must pad your output's leading digits with zeroes.

Code :

```java
import java.util.Scanner;
import java.lang.String;

public class Solution {

    public static void main(String[] args) {
            Scanner sc=new Scanner(System.in);
            System.out.println("================================");
            for(int i=0;i<3;i++){
                String s1=sc.next();
                int x=sc.nextInt();
                System.out.printf("%-15s%03d%n",s1,x);
            }


            System.out.println("================================");

    }
}
```

Output :



6 . Question :

Task

Given an integer, N, print its first 10 multiples. Each multiple $N \times i$ (where $1 \leq i \leq 10$) should be printed on a new line in the form: `N x i = result`.

Input Format

A single integer, N.

Constraints

- $2 \leq N \leq 20$

Output Format

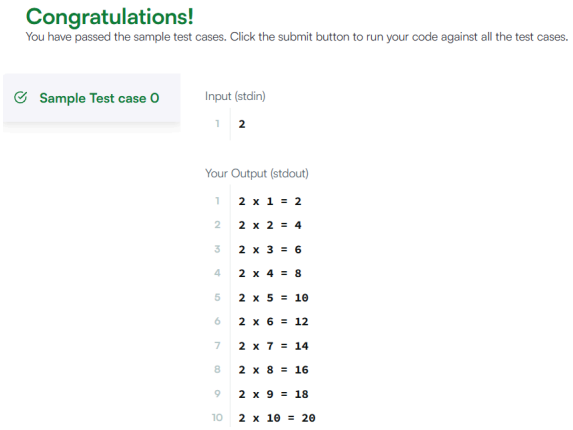
Print 10 lines of output; each line i (where $1 \leq i \leq 10$) contains the $result$ of $N \times i$ in the form:

`N x i = result`.

Code :

```java
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.regex.*;


public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));

        int N = Integer.parseInt(bufferedReader.readLine().trim());

        for (int i=1;i<11;i++){
            int mul=1;
            mul=N*i;
            System.out.println(N+" x "+i+" = "+mul);
        }

        bufferedReader.close();
    }
}
```

Output :



7 . Question :

We use the integers a, b, and n to create the following series:

$$(a + 2^0 \cdot b), (a + 2^0 \cdot b + 2^1 \cdot b), \ldots, (a + 2^0 \cdot b + 2^1 \cdot b + \ldots + 2^{n-1} \cdot b)$$

You are given q queries in the form of a, b, and n. For each query, print the series corresponding to the given a, b, and n values as a single line of n space-separated integers.

Input Format

The first line contains an integer, q, denoting the number of queries.

Each line i of the q subsequent lines contains three space-separated integers describing the respective a_i, b_i, and n_i values for that query.

Constraints

- $0 \leq q \leq 500$
- $0 \leq a, b \leq 50$
- $1 \leq n \leq 15$

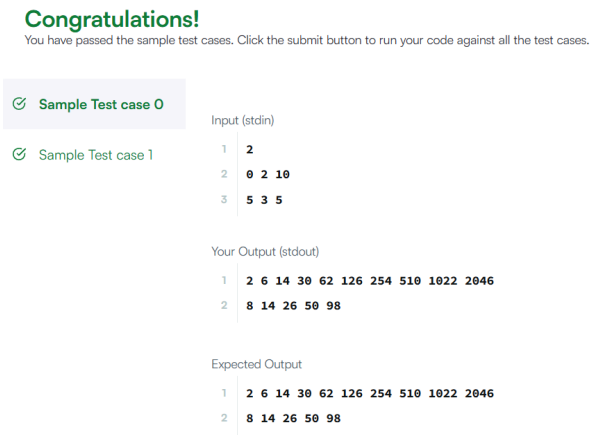
Output Format

For each query, print the corresponding series on a new line. Each series must be printed in order as a single line of n space-separated integers.

Code :

```java
import java.util.*;
import java.io.*;

class Solution{
    public static void main(String []argh){
        Scanner in = new Scanner(System.in);
        int t=in.nextInt();
        for(int i=0;i<t;i++){
            int a = in.nextInt();
            int b = in.nextInt();
            int n = in.nextInt();
            int sum=a;
            for (int j=0;j<n;j++){
                sum+=(1 << j)*b;
                System.out.print(sum + " ");
            }
            System.out.println();
        }
        in.close();
    }
}
```

Output :



8. Question :

Java has 8 primitive data types; char, boolean, byte, short, int, long, float, and double. For this exercise, we'll work with the primitives used to hold integer values (byte, short, int, and long):

- A byte is an 8-bit signed integer.
- A short is a 16-bit signed integer.
- An int is a 32-bit signed integer.
- A long is a 64-bit signed integer.

Given an input integer, you must determine which primitive data types are capable of properly storing that input.

To get you started, a portion of the solution is provided for you in the editor.

Reference: https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html

Input Format

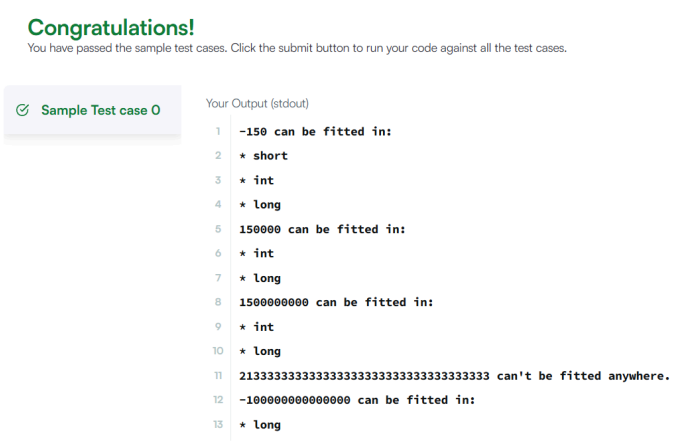
The first line contains an integer, T, denoting the number of test cases.

Each test case, T, is comprised of a single line with an integer, n, which can be arbitrarily large or small.

Code :

```
6   class Solution{
7       public static void main(String []argh)
11
12          Scanner sc = new Scanner(System.in);
13          int t=sc.nextInt();
14
15          for(int i=0;i<t;i++)
16          {
17
18              try
19              {
20                  long x=sc.nextLong();
21                  System.out.println(x+" can be fitted in:");
22                  if(x>=Byte.MIN_VALUE && x<= Byte.MAX_VALUE)System.out.println("* byte");
23                  if(x>=Short.MIN_VALUE && x<=Short.MAX_VALUE)System.out.println("* short");
24                  if(x>=Integer.MIN_VALUE && x<=Integer.MAX_VALUE)System.out.println("* int");
25                  if(x>=Long.MIN_VALUE && x<=Long.MAX_VALUE)System.out.println("* long");
26              }
27              catch(Exception e)
28              {
29                  System.out.println(sc.next()+" can't be fitted anywhere.");
30              }
31
32          }
33      }
34  }
```

Output :



9. Question :

The challenge here is to read n lines of input until you reach EOF, then number and print all n lines of content.

Hint: Java's Scanner.hasNext() method is helpful for this problem.

Input Format

Read some unknown n lines of input from stdin(System.in) until you reach EOF; each line of input contains a non-empty String.

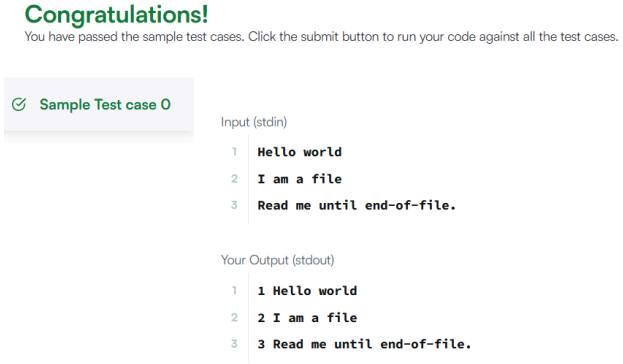
Output Format

For each line, print the line number, followed by a single space, and then the line content received as input.

Code :

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
import java.util.Scanner;

public class Solution {

    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int n=1;
        while (sc.hasNext()){
            String l=sc.nextLine();
            System.out.println(n + " " + l);
            n++;
        }
        sc.close();
    }
}
```

Output :



10. Question :

You are given an integer n, you have to convert it into a string.

Please complete the partially completed code in the editor. If your code successfully converts n into a string s the code will print "Good job". Otherwise it will print "Wrong answer".

n can range between -100 to 100 inclusive.

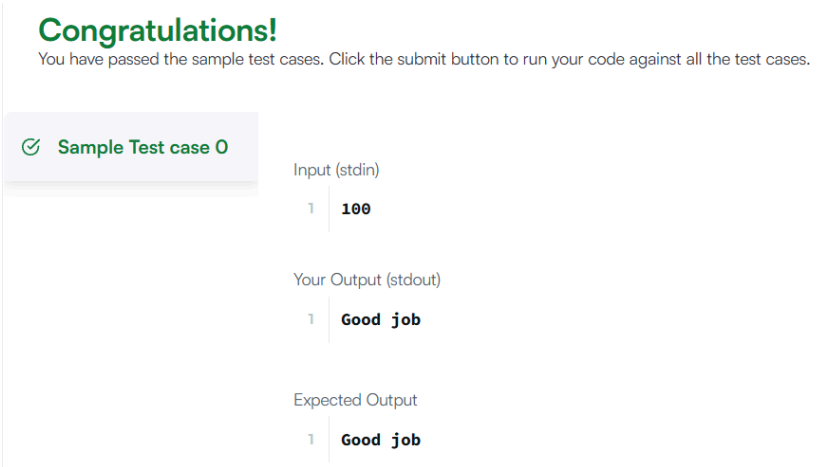
Sample Input 0

```
100
```

Code :

```
4   import java.util.*;…
13    String s= Integer.toString(n);
14
15
16    if (n == Integer.parseInt(s)) {
17     System.out.println("Good job");
18    } else {
19     System.out.println("Wrong answer.");
20    }
21   } catch (DoNotTerminate.ExitTrappedException e) {
22    System.out.println("Unsuccessful Termination!!");
23   }
24  }
25 }
26
27 //The following class will prevent you from terminating the code using exit(0)!
28 class DoNotTerminate {
29
30  public static class ExitTrappedException extends SecurityException {
31
32   private static final long serialVersionUID = 1;
33  }
34
35  public static void forbidExit() {
36   final SecurityManager securityManager = new SecurityManager() {
37    @Override
38    public void checkPermission(Permission permission) {
```

Output :



11 . Question :

Function Description

Complete the findDay function in the editor below.

findDay has the following parameters:

- int: month
- int: day
- int: year

Returns

- string: the day of the week in capital letters

Input Format

A single line of input containing the space separated month, day and year, respectively, in $MM\ DD\ YYYY$ format.

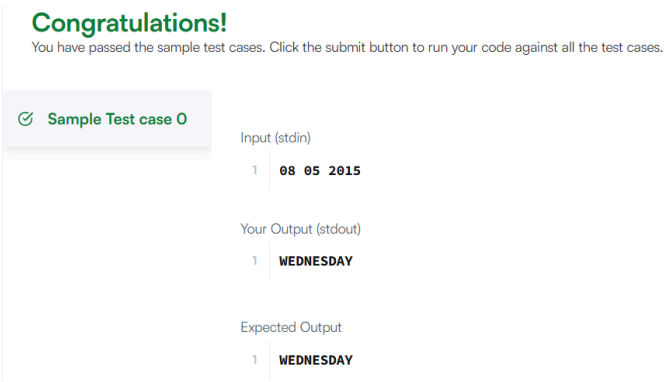
Constraints

- $2000 < year < 3000$

Code :

```java
import java.io.*;···
import java.time.LocalDate;
class Result {

    /*
     * Complete the 'findDay' function below.
     *
     * The function is expected to return a STRING.
     * The function accepts following parameters:
     *  1. INTEGER month
     *  2. INTEGER day
     *  3. INTEGER year
     */

    public static String findDay(int month, int day, int year) {
        java.time.LocalDate date= java.time.LocalDate.of(year , month, day);
        return date.getDayOfWeek().name();

    }
```

Output :



12 . Question :

Input Format

A single double-precision number denoting $payment$.

Constraints

- $0 \leq payment \leq 10^9$

Output Format

On the first line, print `US: u` where u is $payment$ formatted for US currency.

On the second line, print `India: i` where i is $payment$ formatted for Indian currency.

On the third line, print `China: c` where c is $payment$ formatted for Chinese currency.

On the fourth line, print `France: f`, where f is $payment$ formatted for French currency.

Code :

```java
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        double payment = scanner.nextDouble();
        scanner.close();

        Locale usLocale = Locale.US;
        Locale indiaLocale = new Locale("en","IN");
        Locale chinaLocale = Locale.CHINA;
        Locale franceLocale = Locale.FRANCE;

        NumberFormat us=NumberFormat.getCurrencyInstance(usLocale);
        NumberFormat india=NumberFormat.getCurrencyInstance(indiaLocale);
        NumberFormat china=NumberFormat.getCurrencyInstance(chinaLocale);
        NumberFormat france=NumberFormat.getCurrencyInstance(franceLocale);

        System.out.println("US: " + us.format(payment));
        System.out.println("India: " + india.format(payment));
        System.out.println("China: " + china.format(payment));
        System.out.println("France: " + france.format(payment));
    }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

```
12324.134
```

Your Output (stdout)

```
US: $12,324.13
India: Rs.12,324.13
China: ￥12,324.13
France: 12 324,13 €
```

13 . Question :

The elements of a String are called characters. The number of characters in a String is called the length, and it can be retrieved with the String.length() method.

Given two strings of lowercase English letters, A and B, perform the following operations:

1. Sum the lengths of A and B.
2. Determine if A is lexicographically larger than B (i.e.: does B come before A in the dictionary?).
3. Capitalize the first letter in A and B and print them on a single line, separated by a space.

Input Format

The first line contains a string A. The second line contains another string B. The strings are comprised of only lowercase English letters.

Output Format

There are three lines of output:

For the first line, sum the lengths of A and B.

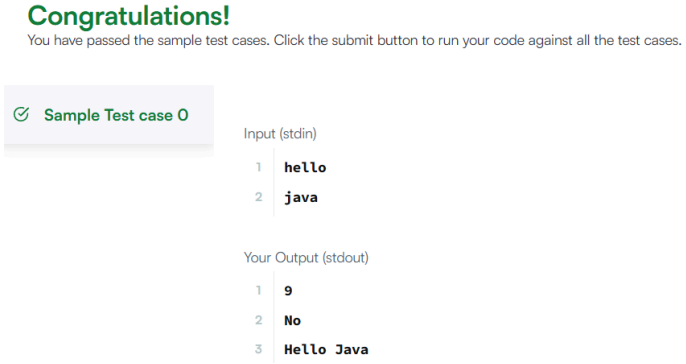
For the second line, write `Yes` if A is lexicographically greater than B otherwise print `No` instead.

For the third line, capitalize the first letter in both A and B and print them on a single line, separated by a space.

Code :

```java
public class Solution {

    public static void main(String[] args) {

        Scanner sc=new Scanner(System.in);
        String A=sc.next();
        String B=sc.next();
        int l1=A.length();
        int l2=B.length();
        System.out.println(l1+l2);
        if (A.compareTo(B)>0){
            System.out.println("Yes");
        }else{
            System.out.println("No");
        }
        String c1=A.substring(0,1).toUpperCase()+A.substring(1);
        String c2=B.substring(0,1).toUpperCase()+B.substring(1);
        System.out.println(c1+" "+c2);

        sc.close();
    }
```

Output :



14 . Question :

Given a string, s, and two indices, $start$ and end, print a substring consisting of all characters in the inclusive range from $start$ to $end - 1$. You'll find the String class' substring method helpful in completing this challenge.

Input Format

The first line contains a single string denoting s.

The second line contains two space-separated integers denoting the respective values of $start$ and end.

Constraints

- $1 \leq |s| \leq 100$
- $0 \leq start < end \leq n$
- String s consists of English alphabetic letters (i.e., $[a - zA - Z]$) only.

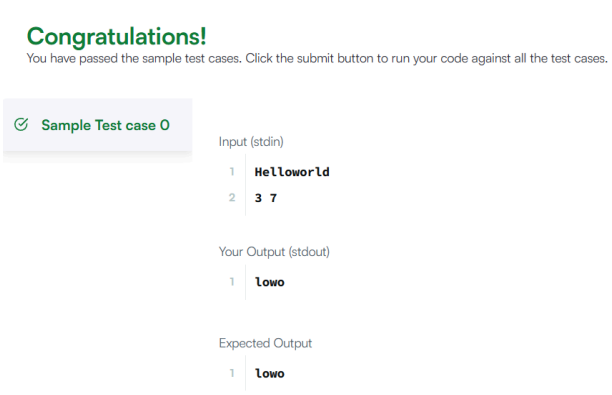
Output Format

Print the substring in the inclusive range from $start$ to $end - 1$.

Code :

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        String S = in.next();
        int start = in.nextInt();
        int end = in.nextInt();
        String str=S.substring(start,end);
        System.out.println(str);
        in.close();
    }
```

Output :



15. Question :

Function Description

Complete the getSmallestAndLargest function in the editor below.

getSmallestAndLargest has the following parameters:

- string s: a string
- int k: the length of the substrings to find

Returns

- string: the string ' + "\n" + ' where and are the two substrings

Input Format

The first line contains a string denoting s.

The second line contains an integer denoting k.

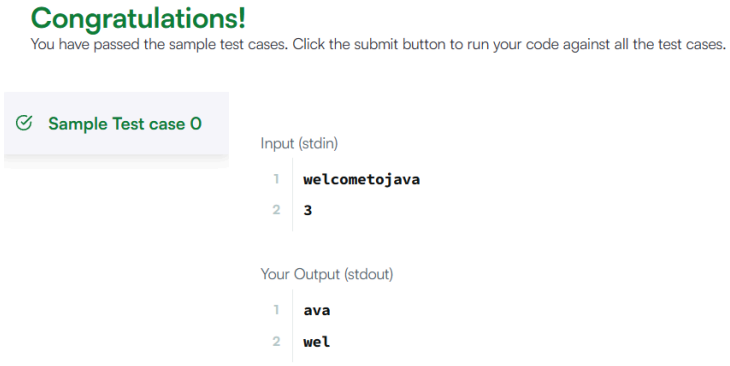
Constraints

- $1 \leq |s| \leq 1000$
- s consists of English alphabetic letters only (i.e., `[a-zA-Z]`).

Code :

```java
import java.util.Scanner;···

    public static String getSmallestAndLargest(String s, int k) {
        String smallest = s.substring(0,k);
        String largest = s.substring(0,k);

        for (int i=1; i<=s.length()-k;i++){
            String cs=s.substring(i,i+k);
            if (cs.compareTo(smallest)<0){
                smallest=cs;
            }

            if (cs.compareTo(largest)>0){
                largest=cs;
            }
        }
        return smallest + "\n" + largest;
    }
```

Output :



16. Question :

A palindrome is a word, phrase, number, or other sequence of characters which reads the same backward or forward.

Given a string A, print `Yes` if it is a palindrome, print `No` otherwise.

Constraints

- A will consist at most 50 lower case english letters.

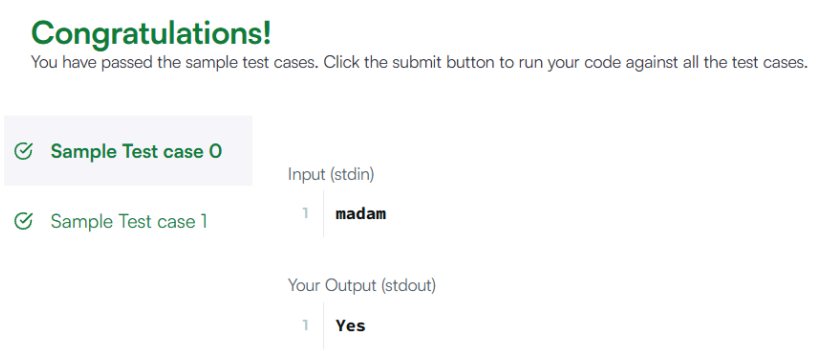
Sample Input

```
madam
```

Code :

```java
import java.io.*;
import java.util.*;

public class Solution {

    public static void main(String[] args) {

        Scanner sc=new Scanner(System.in);
        String A=sc.next();
        String rev="";
        for (int i=A.length()-1;i>=0;i--){
            rev+=A.charAt(i);
        }
        if (A.equals(rev)){
            System.out.println("Yes");
        }else{
            System.out.println("No");
        }

    }
```

Output :



17 . Question :

Input Format

There are two lines of input. The first line contains B: the breadth of the parallelogram. The next line contains H: the height of the parallelogram.

Constraints

- $-100 \leq B \leq 100$
- $-100 \leq H \leq 100$

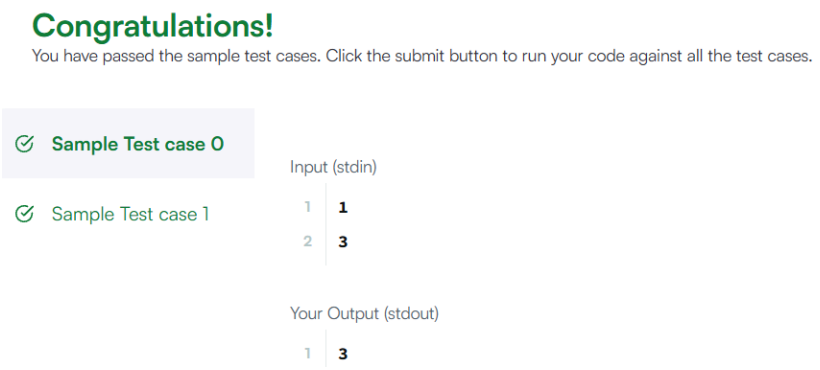
Output Format

If both values are greater than zero, then the main method must output the area of the parallelogram. Otherwise, print "java.lang.Exception: Breadth and height must be positive" without quotes.

Code :

```java
import java.io.*;
import java.util.*;
import java.util.Scanner;

public class Solution {

    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int b=sc.nextInt();
        int h=sc.nextInt();
        if (b>0 && h>0){
            int a=b*h;
            System.out.println(a);
        }else{
            System.out.println("java.lang.Exception: Breadth and height must be positive");
        }
```

Output :



18 . Question :

Complete the isAnagram function in the editor.

isAnagram has the following parameters:

- string a: the first string
- string b: the second string

Returns

- boolean: If a and b are case-insensitive anagrams, return true. Otherwise, return false.

Input Format

The first line contains a string a.

The second line contains a string b.

Code :

```
1  > import java.util.Scanner; ···
4
5      static boolean isAnagram(String a, String b) {
6          if (a==null || b==null || a.length()!=b.length()){
7              return false;
8          }
9          a=a.toLowerCase();
10         b=b.toLowerCase();
11         int [] f=new int[26];
12         for (int i=0;i<a.length();i++){
13             f[a.charAt(i)-'a']++;
14             f[b.charAt(i)-'a']--;
15         }
16
17         for (int count : f){
18             if (count != 0){
19                 return false;
20             }
21         }
22         return true;
23     }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

- Sample Test case 0
- Sample Test case 1
- Sample Test case 2

Input (stdin)

```
1  anagram
2  margana
```

Your Output (stdout)

```
1  Anagrams
```

19 . Question :

Input Format

A single string, s.

Constraints

- $1 \leq$ length of $s \leq 4 \cdot 10^5$
- s is composed of any of the following: English alphabetic letters, blank spaces, exclamation points (!), commas (,), question marks (?), periods (.), underscores (_), apostrophes ('), and at symbols (@).

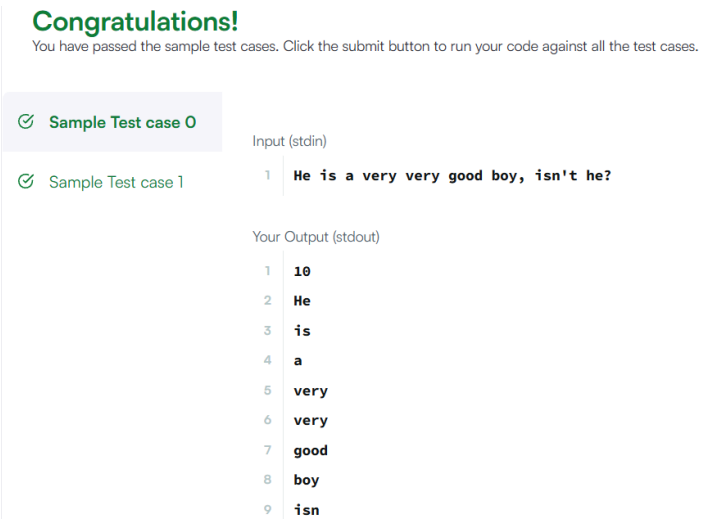
Output Format

On the first line, print an integer, n, denoting the number of tokens in string s (they do not need to be unique). Next, print each of the n tokens on a new line in the same order as they appear in input string s.

Code :

```java
import java.io.*;
import java.util.*;

public class Solution {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        String s = scan.nextLine();
        s=s.trim();
        if (s.length()==0){
            System.out.println(0);
            return;
        }

        String [] ts = s.split("[^A-Za-z]+");
        System.out.println(ts.length);
        for (String t : ts){
            System.out.println(t);
        }

        scan.close();
    }
}
```

Output :



20. Question :

Input Format

The first line of input contains an integer N, denoting the number of test cases. The next N lines contain a string of any printable characters representing the pattern of a regex.

Output Format

For each test case, print `Valid` if the syntax of the given pattern is correct. Otherwise, print `Invalid`. Do not print the quotes.

Code :

```java
import java.util.Scanner;
import java.util.regex.*;

public class Solution
{
    public static void main(String[] args){
        Scanner in = new Scanner(System.in);
        int testCases = Integer.parseInt(in.nextLine());
        while(testCases>0){
            String pattern = in.nextLine();
            try{
                Pattern.compile(pattern);
                System.out.println("Valid");
            }catch (PatternSyntaxException e){
                System.out.println("Invalid");
            }
            testCases--;
        }
        in.close();
    }
}
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

```
3
([A-Z])(.+)
[AZ[a-z](a-z)
batcatpat(nat
```

Your Output (stdout)

```
Valid
Invalid
Invalid
```

21 . Code :

```java
import java.util.regex.Matcher; ···
class MyRegex{
    String n="(\\d{1,2}|(0|1)\\d{2}|2[0-4]\\d|25[0-5])";
    public String pattern = n+"\\."+n+"\\."+n+"\\."+n;
}
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

```
000.12.12.034
121.234.12.12
23.45.12.56
00.12.123.123123.123
122.23
Hello.IP
```

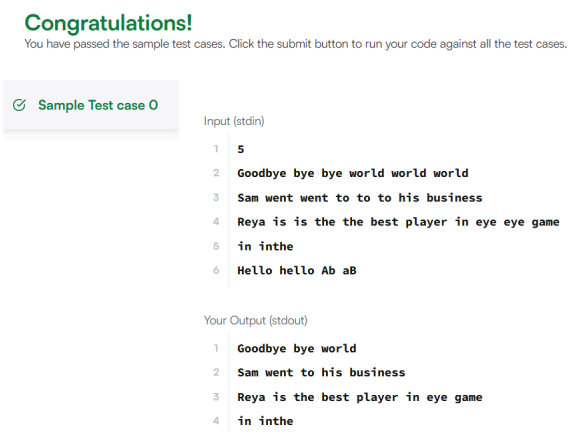
Your Output (stdout)

```
true
true
true
false
```

22. Code :

```java
public class DuplicateWords {

    public static void main(String[] args) {

        String regex = "\\b(\\w+)(?:\\s+\\1)+\\b";
        Pattern p = Pattern.compile(regex, Pattern.CASE_INSENSITIVE);

        Scanner in = new Scanner(System.in);
        int numSentences = Integer.parseInt(in.nextLine());

        while (numSentences-- > 0) {
            String input = in.nextLine();

            Matcher m = p.matcher(input);

            // Check for subsequences of input that match the compiled pattern
            while (m.find()) {
                input = input.replaceAll(m.group(),m.group(1));
            }

            // Prints the modified sentence.
            System.out.println(input);
        }
```

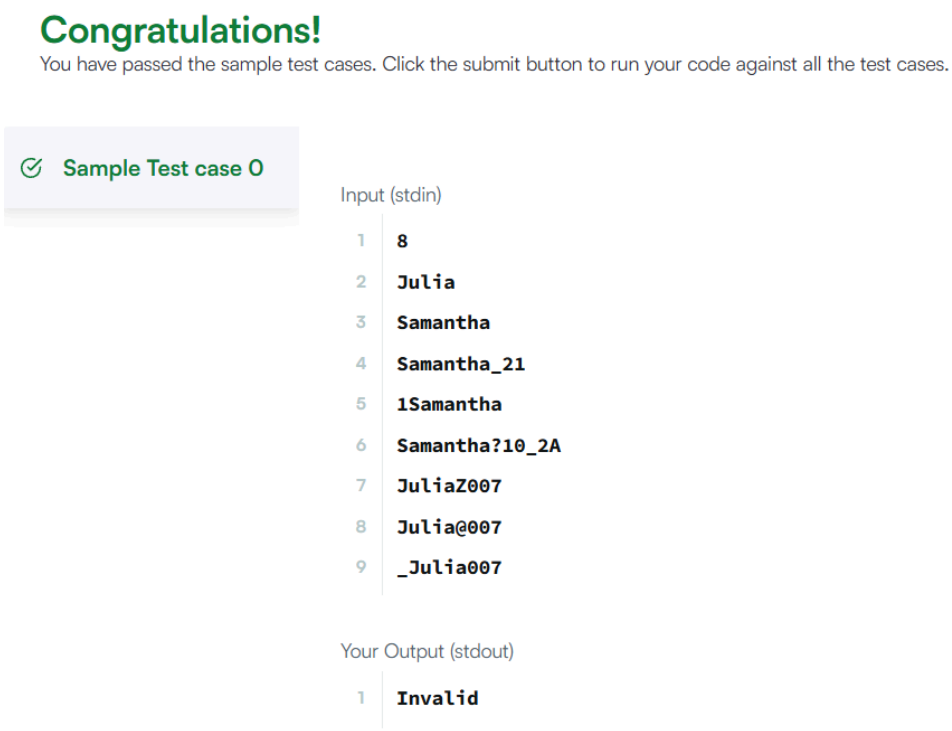
Output :



23 . Code :

```java
import java.util.Scanner;
class UsernameValidator {
    /*
     * Write regular expression here.
     */
    public static final String regularExpression = "^[a-zA-Z][a-zA-Z0-9_]{7,29}$";
}
```

Output :



24 . Code :

```java
public class Solution{
    public static void main(String[] args){

        Scanner in = new Scanner(System.in);
        int testCases = Integer.parseInt(in.nextLine());
        String regex ="<([^>]+)>([^<]+)</\\1>";
        Pattern pattern = Pattern.compile(regex);
        while(testCases>0){
            String line = in.nextLine();

            Matcher matcher=pattern.matcher(line);
            boolean found=false;
            while (matcher.find()){
                System.out.println(matcher.group(2));
                found=true;
            }
            if (!found){
                System.out.println("None");
            }

            testCases--;
        }
    }
}
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

- Sample Test case 0
- Sample Test case 1
- Sample Test case 2

Input (stdin) Download

```
4
<h1>Nayeem loves counseling</h1>
<h1><h1>Sanjay has no watch</h1></h1><par>So wait for a while<par>
<Amee>safat codes like a ninja</amee>
<SA premium>Imtiaz has a secret crash</SA premium>
```

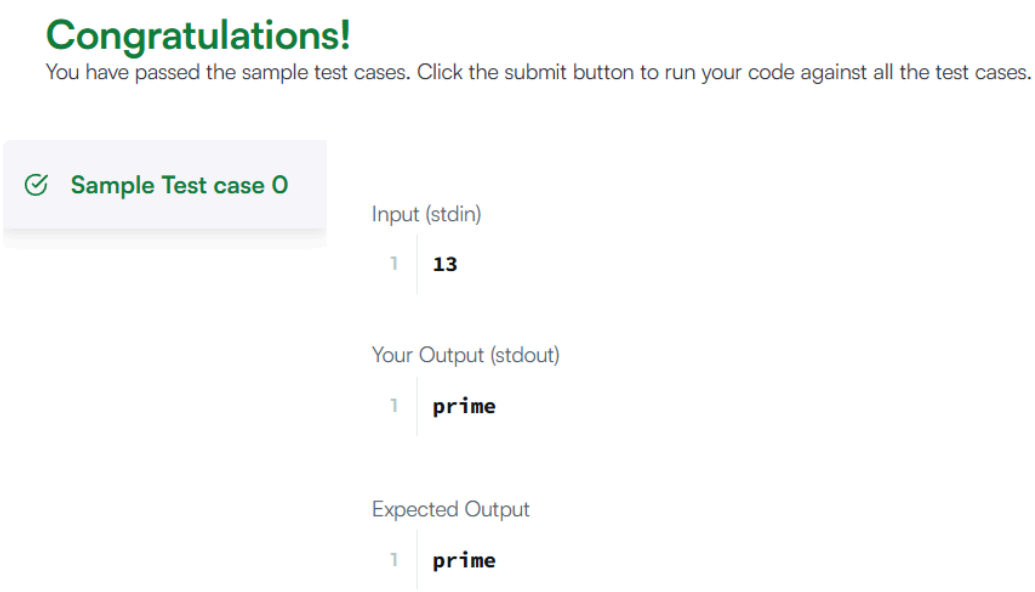
Your Output (stdout)

```
Nayeem loves counseling
Sanjay has no watch
None
Imtiaz has a secret crash
```

25 . Code :

```java
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.regex.*;



public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));

        String n = bufferedReader.readLine().trim();
        BigInteger number = new BigInteger(n);
        if (number.isProbablePrime(1)){
            System.out.println("prime");
        }else{
            System.out.println("not prime");
        }

        bufferedReader.close();
    }
```

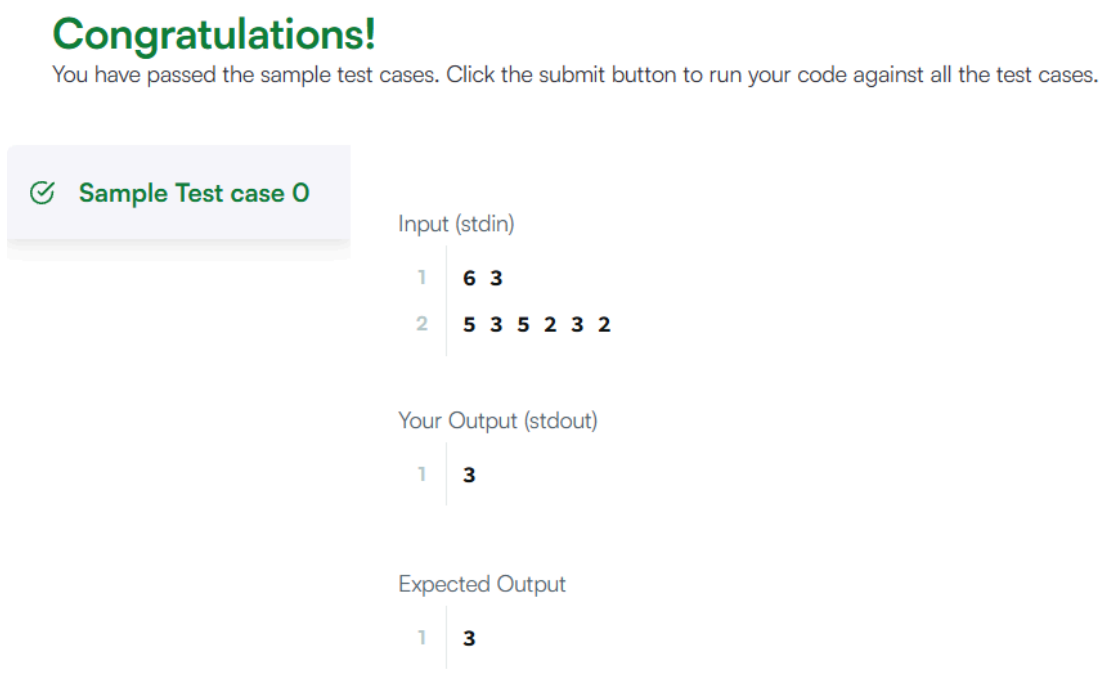
Output :



26. Code :

```java
import java.util.*;
public class test {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        Deque deque = new ArrayDeque<>();
        Set set = new HashSet<>();
        int n = in.nextInt();
        int m = in.nextInt();
        int maxUnique = 0;

        for (int i = 0; i < n; i++) {
            int num = in.nextInt();
            deque.add(num);
            set.add(num);

            if (deque.size()==m){
                if (set.size() > maxUnique){
                    maxUnique = set.size();
                }
                if (maxUnique == m){
                    break;
                }
                int removed = (int) deque.removeFirst();
                if (!deque.contains(removed)){
                    set.remove(removed);
                }
            }
```

Output :



27. Code :

```
7   public class Solution {
9       public static void main(String[] args) {
17          for (int i=0; i<m; i++){
18              String op = sc.next();
19              int p1 = sc.nextInt();
20              int p2 = sc.nextInt();
21
22              switch (op){
23                  case "AND":
24                      bitsets[p1].and(bitsets[p2]);
25                      break;
26                  case "OR":
27                      bitsets[p1].or(bitsets[p2]);
28                      break;
29                  case "XOR":
30                      bitsets[p1].xor(bitsets[p2]);
31                      break;
32                  case "FLIP":
33                      bitsets[p1].flip(p2);
34                      break;
35                  case "SET":
36                      bitsets[p1].set(p2);
37                      break;
38              }
39              System.out.println(bitsets[1].cardinality()+" "+bitsets[2].cardinality());
40          }
41          sc.close();
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

```
5 4
AND 1 2
SET 1 4
FLIP 2 2
OR 2 1
```

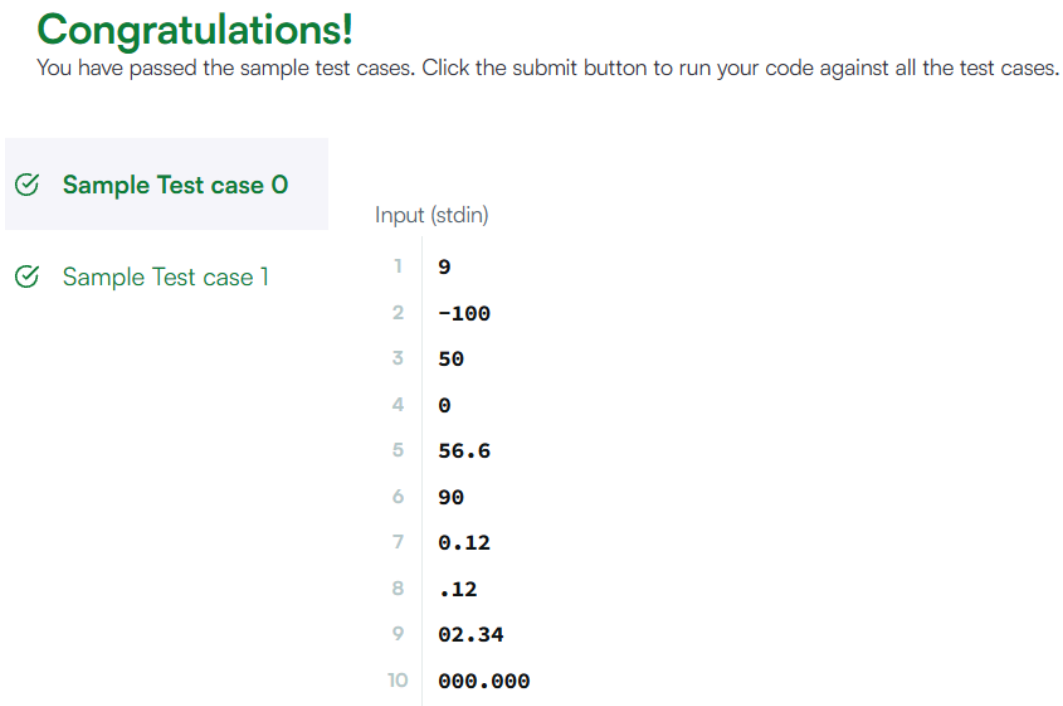
Your Output (stdout)

```
0 0
1 0
1 1
1 2
```

28 . Code :

```
5    import java.math.BigDecimal;···
14
15           // Sort array 's' of size 'n' in descending order
16       Arrays.sort(s, 0, n, new Comparator<String>() {
17           @Override
18           public int compare(String s1, String s2) {
19               BigDecimal b1 = new BigDecimal(s1);
20               BigDecimal b2 = new BigDecimal(s2);
21
22               // Compare b2 to b1 for descending order
23               return b2.compareTo(b1);
24       }
25   });
26 >         //Output···
```

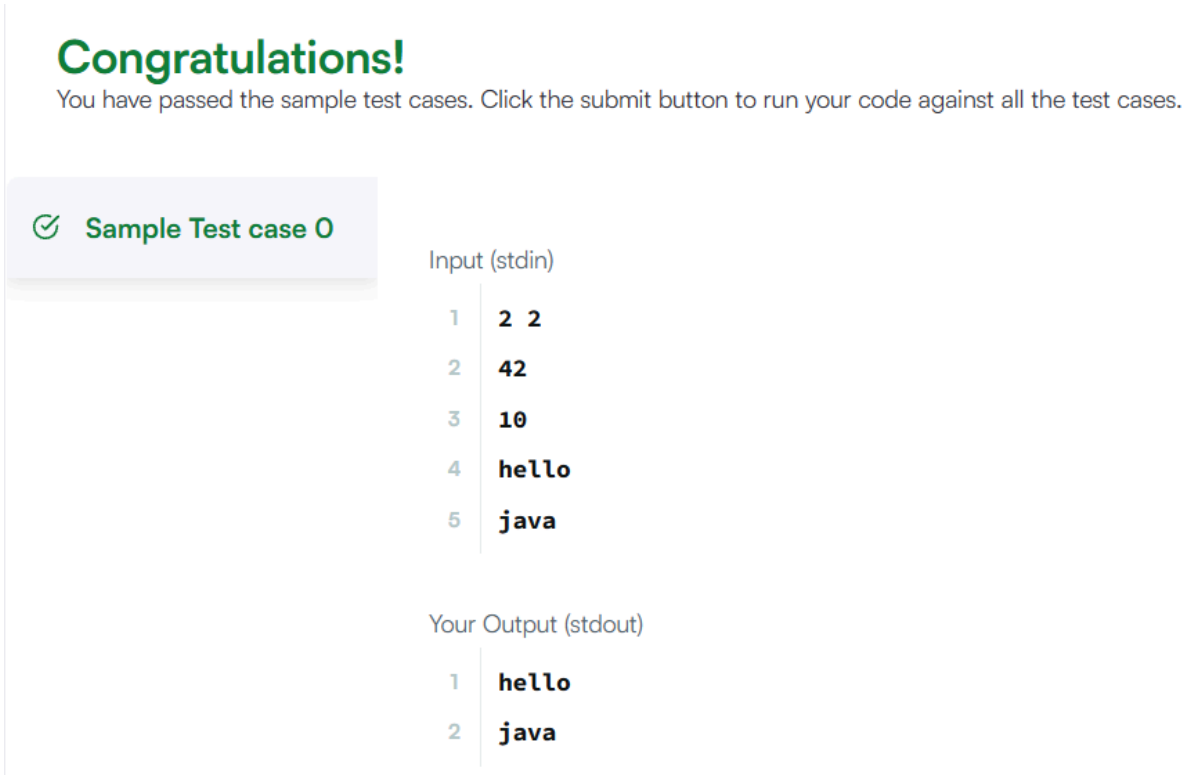
Output :



29 . Code :

```java
import java.util.*;
public class Main{

   static Iterator func(ArrayList mylist){
      Iterator it=mylist.iterator();
      while(it.hasNext()){
         Object element = it.next();
         if(element instanceof String)

            break;
        }
      return it;

   }
   @SuppressWarnings({ "unchecked" })
   public static void main(String []args){
      ArrayList mylist = new ArrayList();
      Scanner sc = new Scanner(System.in);
      int n = sc.nextInt();
      int m = sc.nextInt();
      for(int i = 0;i<n;i++){
         mylist.add(sc.nextInt());
      }

      mylist.add("###");
      for(int i=0;i<m;i++){
         mylist.add(sc.next());
```

Output :



30. Code :

```java

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));

        int[][] arr = new int[6][6];

        // Read the 6x6 2D array
        for (int i = 0; i < 6; i++) {
            String[] rowItems = bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");
            for (int j = 0; j < 6; j++) {
                arr[i][j] = Integer.parseInt(rowItems[j]);
            }
        }

        // Initialize max sum to minimum possible integer value
        int maxHourglassSum = Integer.MIN_VALUE;

        // Iterate through top-left corners of all possible 3x3 hourglasses
        for (int i = 0; i <= 3; i++) {
            for (int j = 0; j <= 3; j++) {
                // Sum the 7 elements forming the hourglass
                int currentSum = arr[i][j]   + arr[i][j+1]   + arr[i][j+2]
                                             + arr[i+1][j+1]
                               + arr[i+2][j] + arr[i+2][j+1] + arr[i+2][j+2];

```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

```
1 1 1 0 0 0
0 1 0 0 0 0
1 1 1 0 0 0
0 9 2 -4 -4 0
0 0 0 -2 0 0
0 0 -1 -2 -4 0
```

Your Output (stdout)

```
13
```

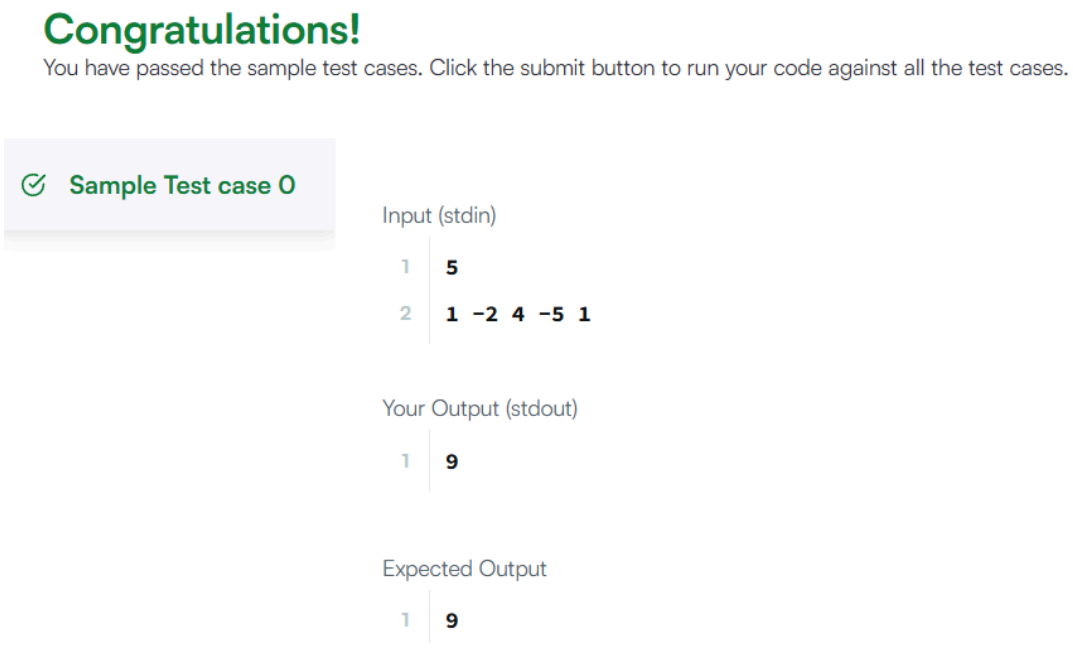
Expected Output

```
13
```

31 . Code :

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args){
        Scanner scanner = new Scanner(System.in);

        // Read size of array
        int n = scanner.nextInt();
        int[] arr = new int[n];

        // Read array elements
        for (int i = 0; i < n; i++) {
            arr[i] = scanner.nextInt();
        }

        int negativeSubarrayCount = 0;

        // Loop through every possible starting index 'i'
        for (int i = 0; i < n; i++) {
            int currentSum = 0;

```

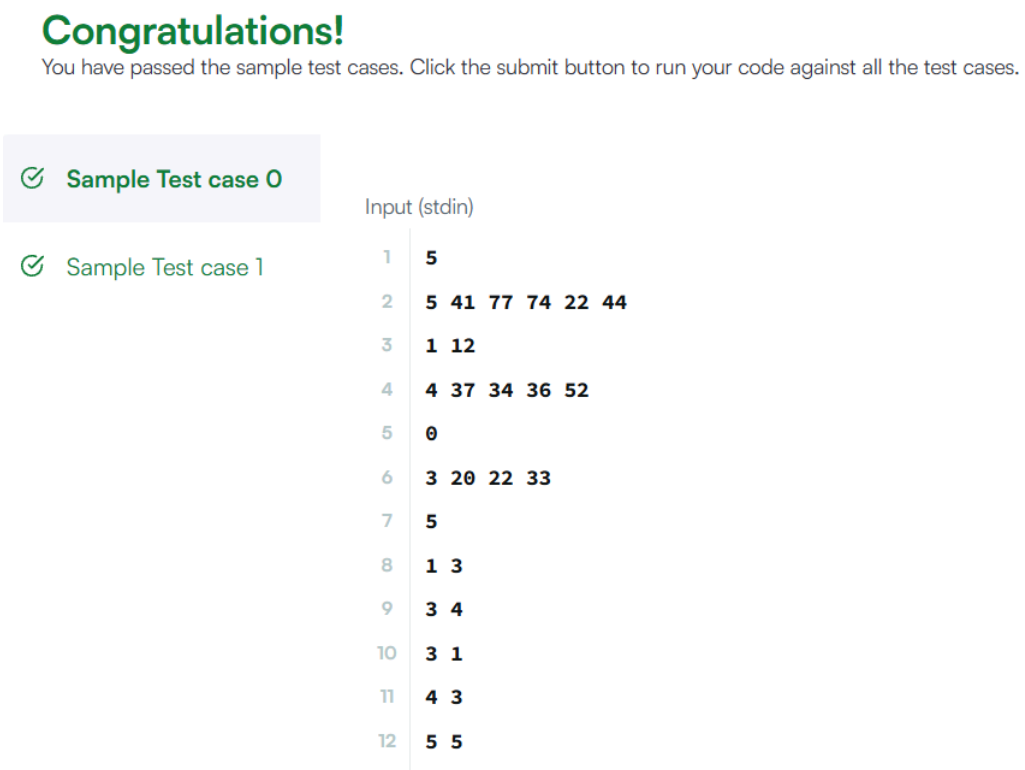
Output :



32 . Code :

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Read the number of lines 'n'
        int n = scanner.nextInt();

        // 2D ArrayList to store rows of varying lengths
        ArrayList<ArrayList<Integer>> lines = new ArrayList<>();

        // Read each line
        for (int i = 0; i < n; i++) {
            int d = scanner.nextInt(); // Number of elements in current line
            ArrayList<Integer> currentLine = new ArrayList<>();

            for (int j = 0; j < d; j++) {
                currentLine.add(scanner.nextInt());
            }

            lines.add(currentLine);
```

Output :



33 . Code :

```java
import java.util.*;

public class Solution {

    public static boolean canWin(int leap, int[] game) {
        return isSolvable(leap, game, 0);
    }

    private static boolean isSolvable(int leap, int[] game, int i) {
        if (i >= game.length) {
            return true;
        }

        if (i < 0 || game[i] == 1) {
            return false;
        }
        game[i] = 1;
        return isSolvable(leap, game, i + leap)
            || isSolvable(leap, game, i + 1)
            || isSolvable(leap, game, i - 1);
    }

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int q = scan.nextInt();
        while (q-- > 0) {
            int n = scan.nextInt();
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

```
4
5 3
0 0 0 0 0
6 5
0 0 0 1 1 1
6 3
0 0 1 1 1 0
3 1
0 1 0
```

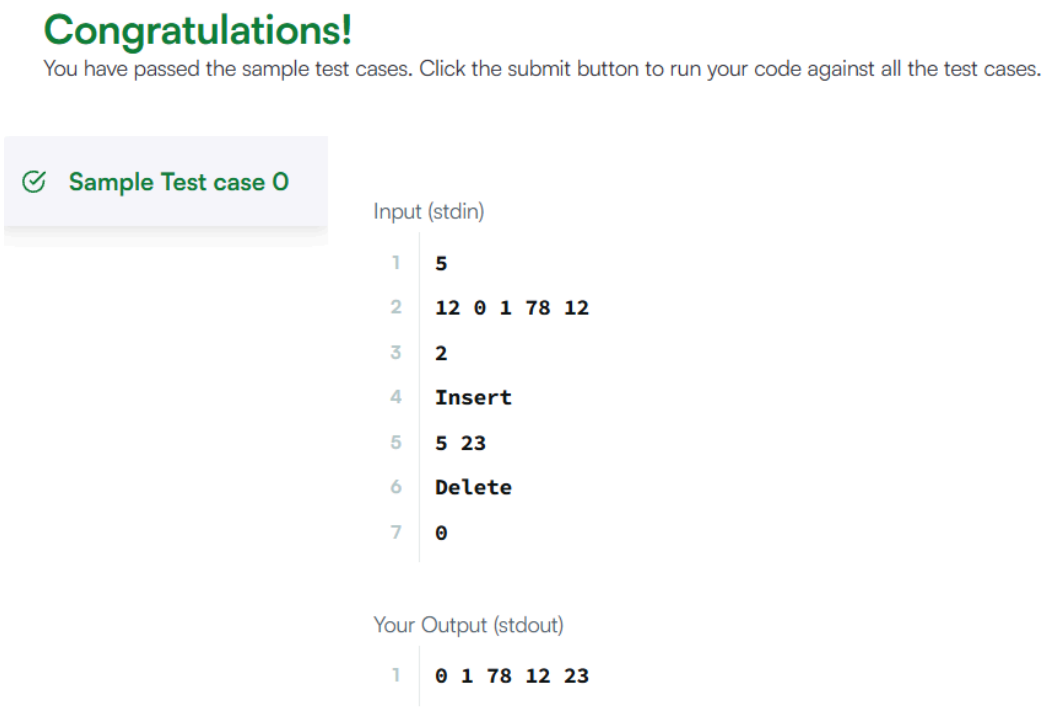
Your Output (stdout)

```
YES
```

34 . Code :

```java
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Read initial list size N
        int n = scanner.nextInt();

        // Initialize dynamic List L
        List<Integer> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            list.add(scanner.nextInt());
        }

        // Read number of queries Q
        int q = scanner.nextInt();

        // Process each query
        for (int i = 0; i < q; i++) {
            String queryType = scanner.next();
```

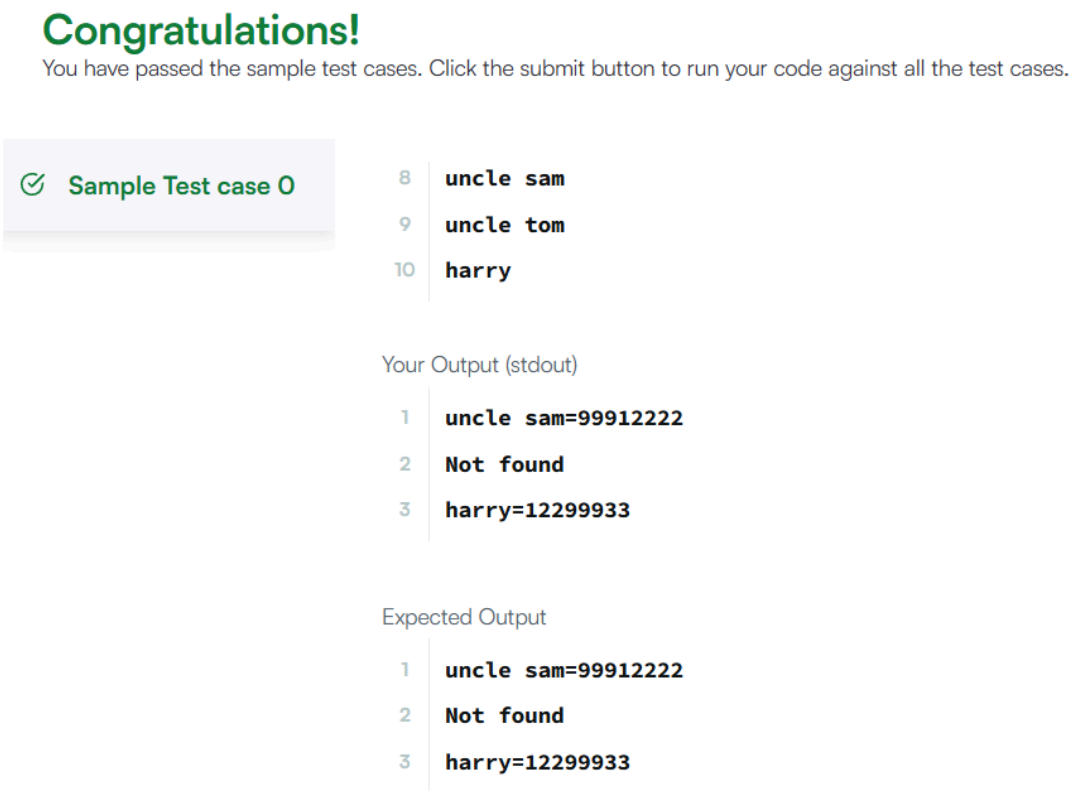
Output :



35 . Code :

```java
//Complete this code or write your own from scratch
import java.util.*;
import java.io.*;

class Solution{
    public static void main(String []argh)
    {
        Scanner in = new Scanner(System.in);

        int n = in.nextInt();
        in.nextLine();

        Map<String, String> phoneBook = new HashMap<>();


        for (int i = 0; i < n; i++) {
            String name = in.nextLine();
            String phone = in.nextLine();
            phoneBook.put(name, phone);
        }


        while (in.hasNext()) {
            String query = in.nextLine();

            if (phoneBook.containsKey(query)) {
                System.out.println(query + "=" + phoneBook.get(query));
```

Output :



36 . Code :

```
class Solution{
    public static void main(String []argh)
        while (sc.hasNext()) {
            String input=sc.next();
            Deque<Character> stack = new ArrayDeque<>();
            boolean isBalanced = true;

            for (int i = 0; i < input.length(); i++) {
                char ch = input.charAt(i);

                if (ch == '(' || ch == '{' || ch == '[') {
                    stack.push(ch);
                } else {
                    if (stack.isEmpty()) {
                        isBalanced = false;
                        break;
                    }

                    char top = stack.pop();
                    if ((ch == ')' && top != '(') ||
                        (ch == '}' && top != '{') ||
                        (ch == ']' && top != '[')) {
                        isBalanced = false;
                        break;
                    }
                }
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

```
{}()
({()})
{}(
[]
```

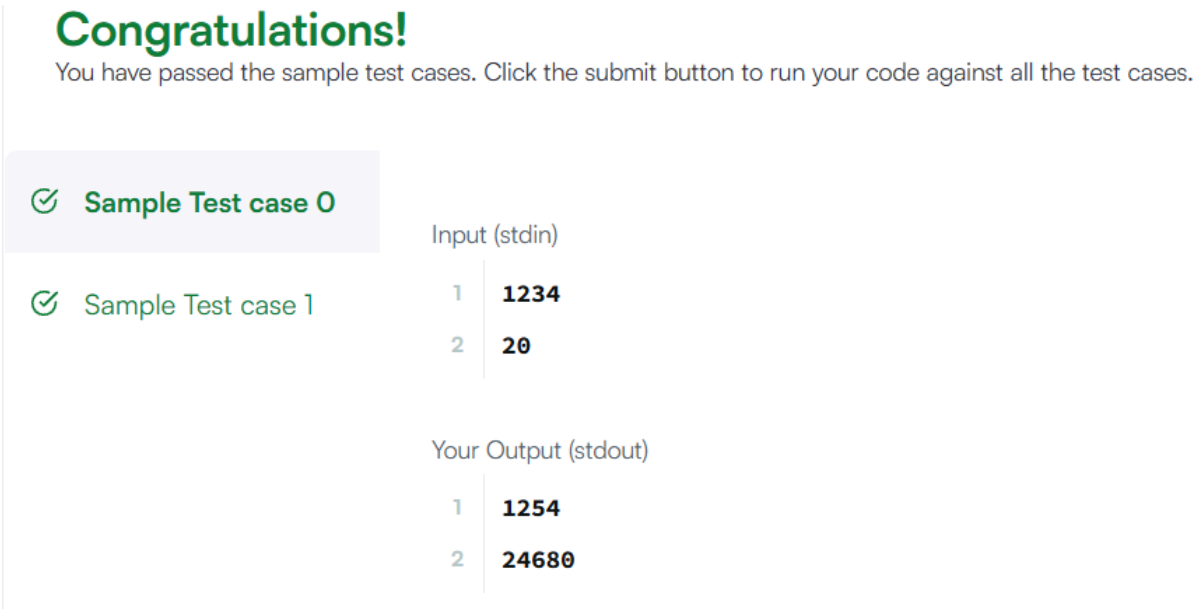
Your Output (stdout)

```
true
true
false
true
```

37 . Code :

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Read input strings as BigInteger
        BigInteger a = new BigInteger(sc.next());
        BigInteger b = new BigInteger(sc.next());

        // Compute sum and product
        BigInteger sum = a.add(b);
        BigInteger product = a.multiply(b);

        // Output results
        System.out.println(sum);
        System.out.println(product);

        sc.close();
    }
```

Output :



38 . Code :

```java
import java.util.*;

public class Solution {

    public static void main(String[] args) {

        Scanner scan = new Scanner(System.in);
        int n = scan.nextInt();
        int[] a = new int[n];
        for (int i=0; i<n; i++){
            a[i] = scan.nextInt();
        }
        scan.close();

        // Prints each sequential element in array a
        for (int i = 0; i < a.length; i++) {
            System.out.println(a[i]);
        }
    }
}
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

```
5
10
20
30
40
50
```

Your Output (stdout)

```
10
20
30
40
```

39. Code :

```java

import java.io.IOException;
import java.lang.reflect.Method;

class Printer
{
   public <E> void printArray(E[] array){
    for (E element : array){
        System.out.println(element);
    }
   }

}
public class Solution {


    public static void main( String args[] ) {
        Printer myPrinter = new Printer();
        Integer[] intArray = { 1, 2, 3 };
        String[] stringArray = {"Hello", "World"};
        myPrinter.printArray(intArray);
        myPrinter.printArray(stringArray);
        int count = 0;

        for (Method method : Printer.class.getDeclaredMethods()) {
            String name = method.getName();
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
1
2
3
Hello
World
```

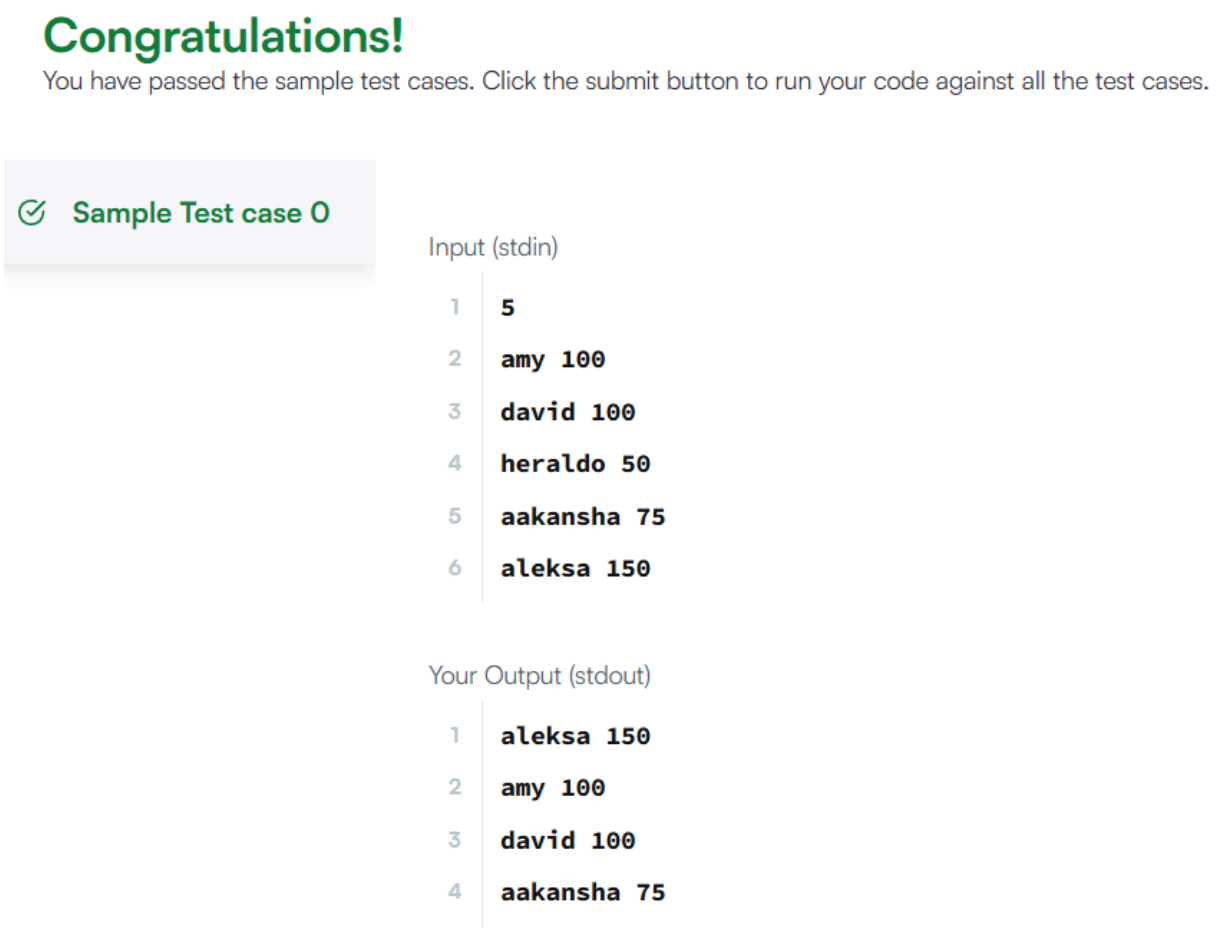
Expected Output

```
1
2
3
Hello
World
```

40 . Code :

```java
import java.util.*;
class Checker implements Comparator<Player> {
    @Override
    public int compare(Player a, Player b) {
        if (a.score == b.score) {
            return a.name.compareTo(b.name);
        }
        return Integer.compare(b.score, a.score); // Descending order
    }
}


class Player{ ···
```

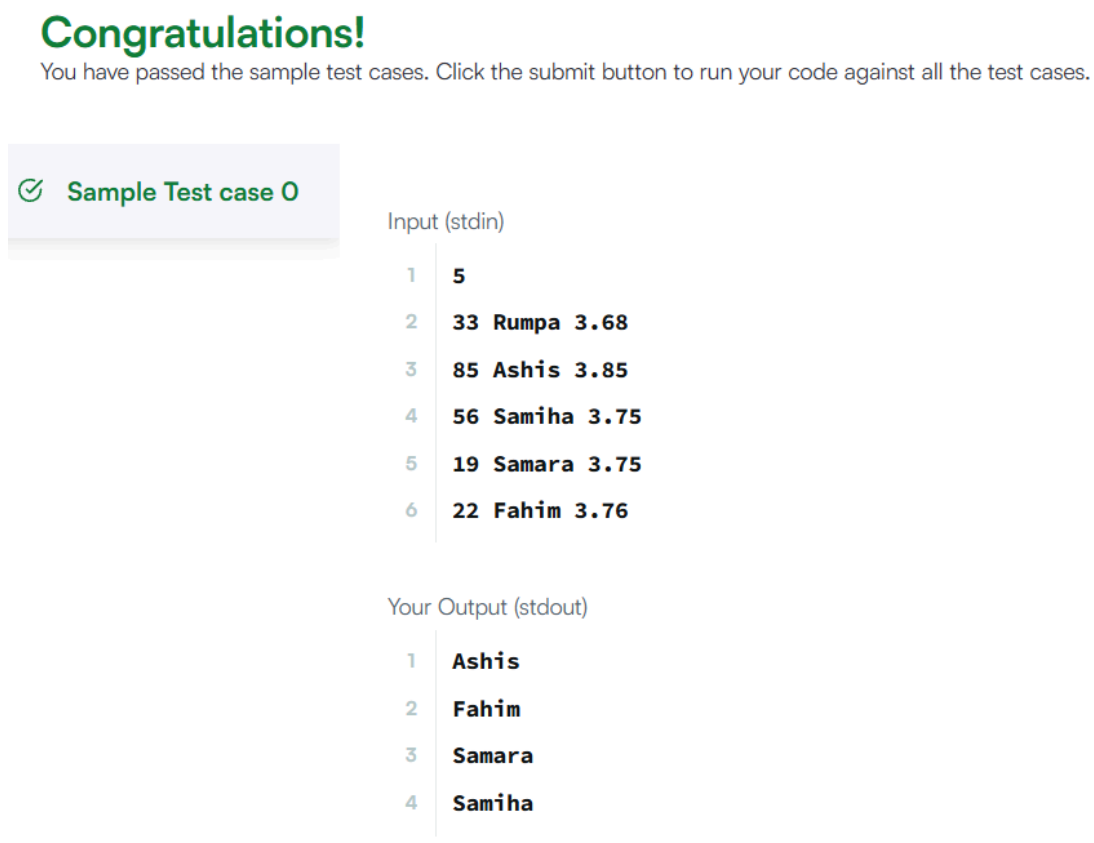
Output :



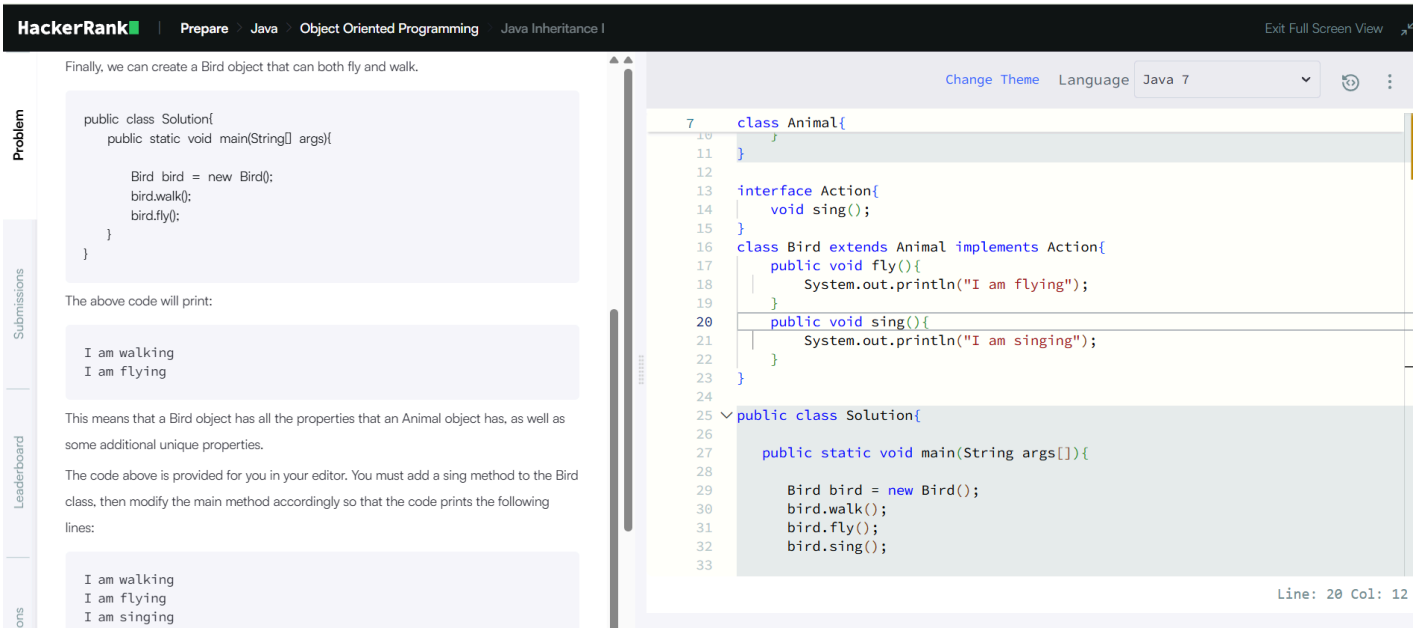
41 . Code:

```
25      public class Solution
27          public static void main(String[] args){
42
43              Collections.sort(studentList, new Comparator<Student>() {
44
45                  public int compare(Student s1, Student s2) {
46
47                      if (s1.getCgpa() != s2.getCgpa()) {
48                          return Double.compare(s2.getCgpa(), s1.getCgpa());
49                      }
50
51                      if (!s1.getFname().equals(s2.getFname())) {
52                          return s1.getFname().compareTo(s2.getFname());
53                      }
54
55                      return Integer.compare(s1.getId(), s2.getId());
56                  }
57              });
58
59              for(Student st: studentList){
60                  System.out.println(st.getFname());
61              }
62          }
63      }
```

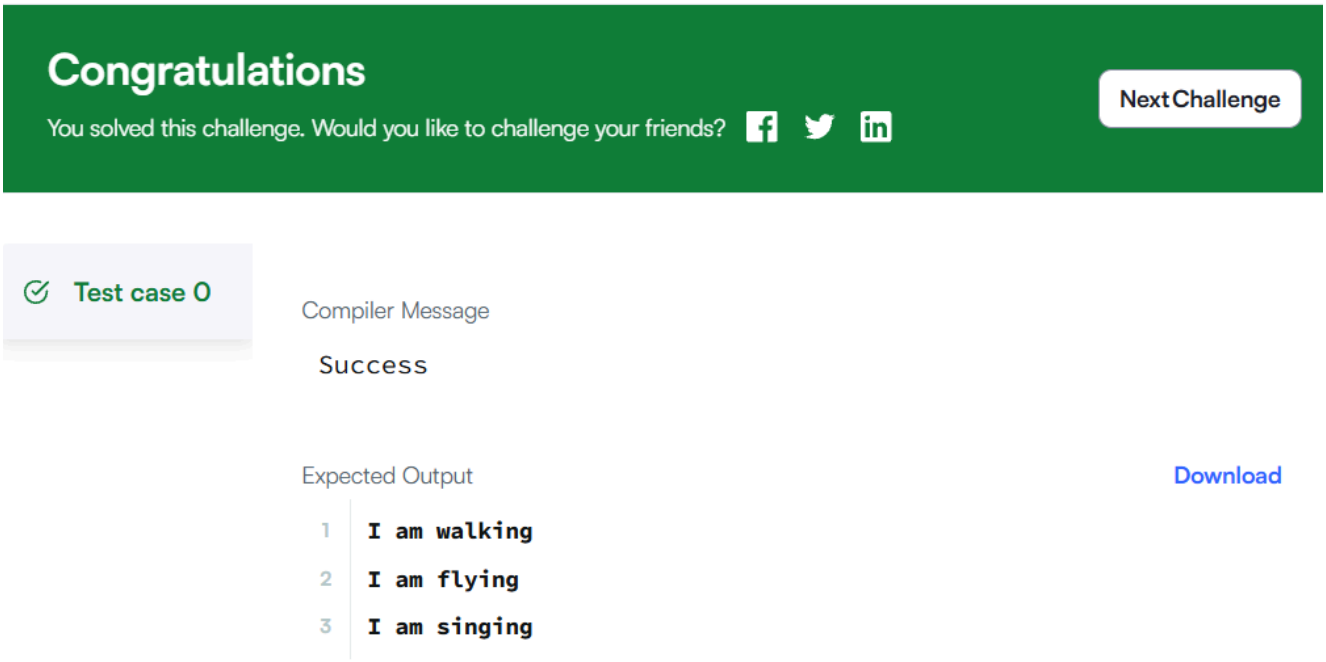
Output :



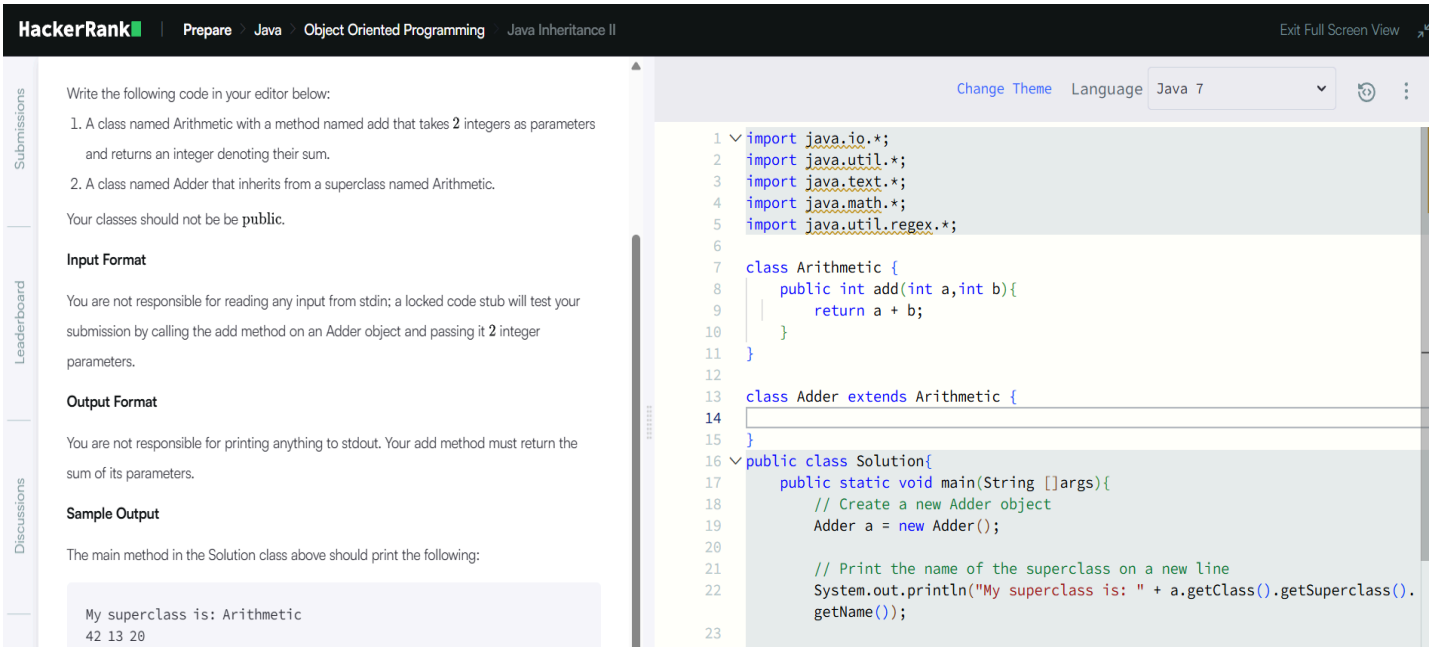
42 . Code :



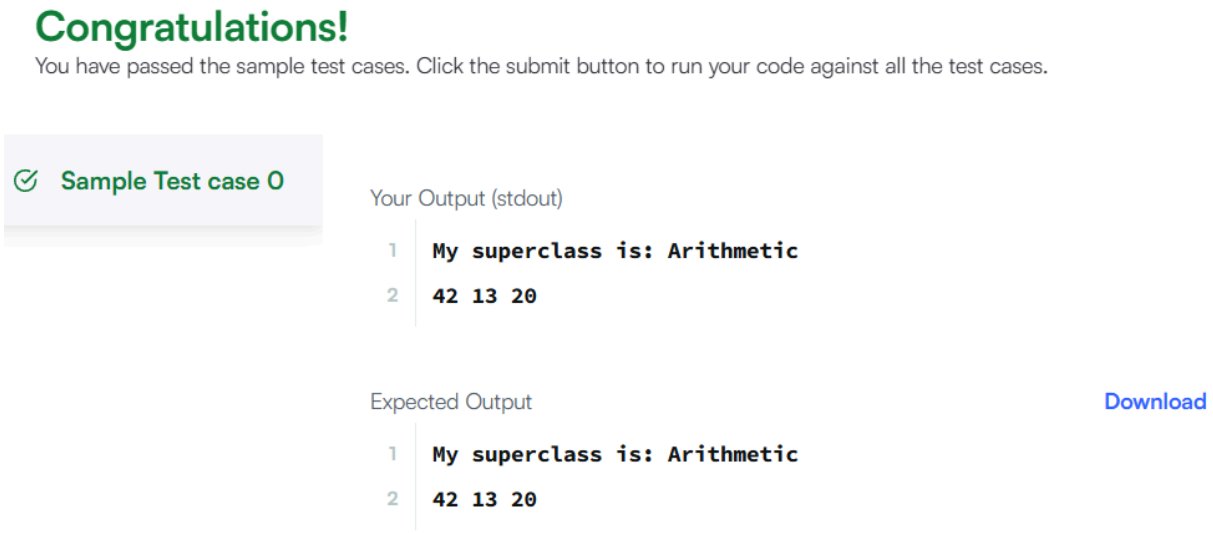
Output :



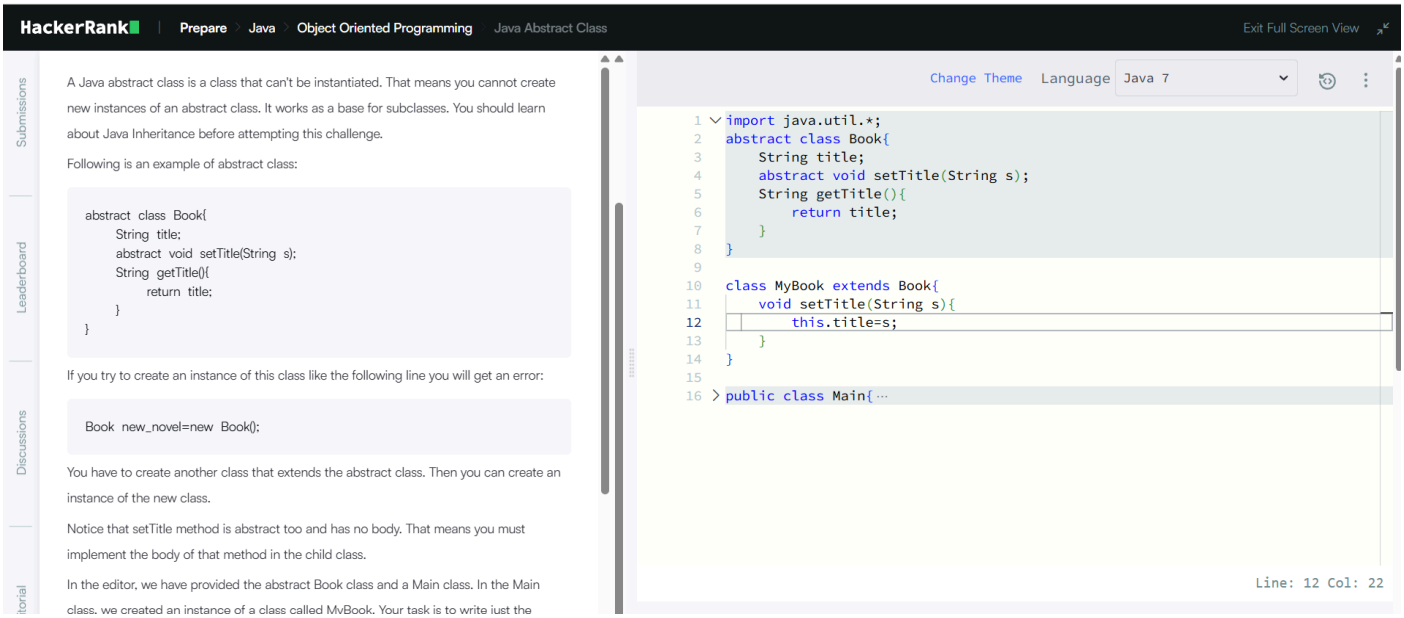
43 . Code :



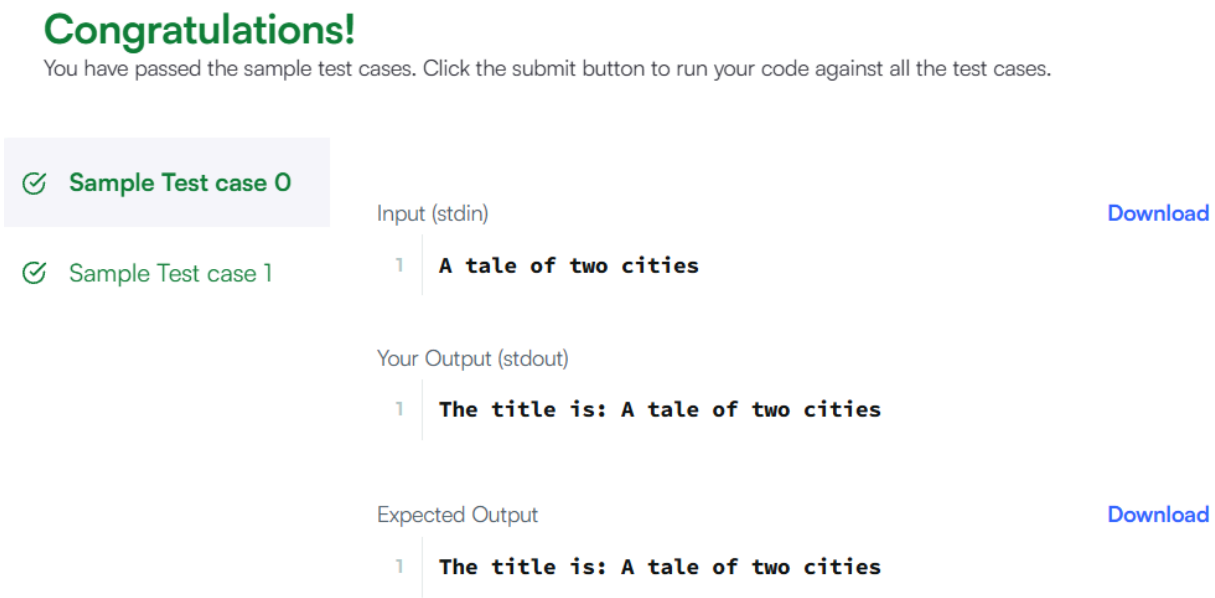
Output :



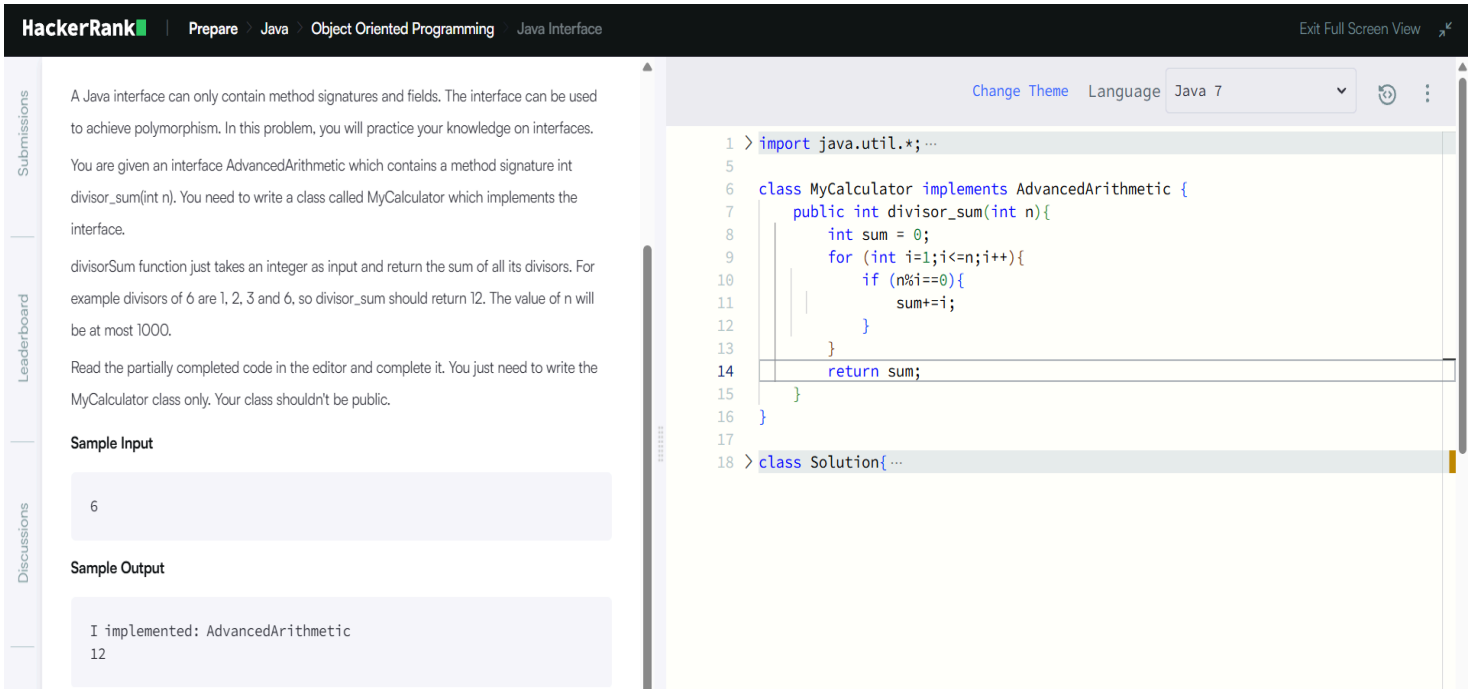
44 . Code :



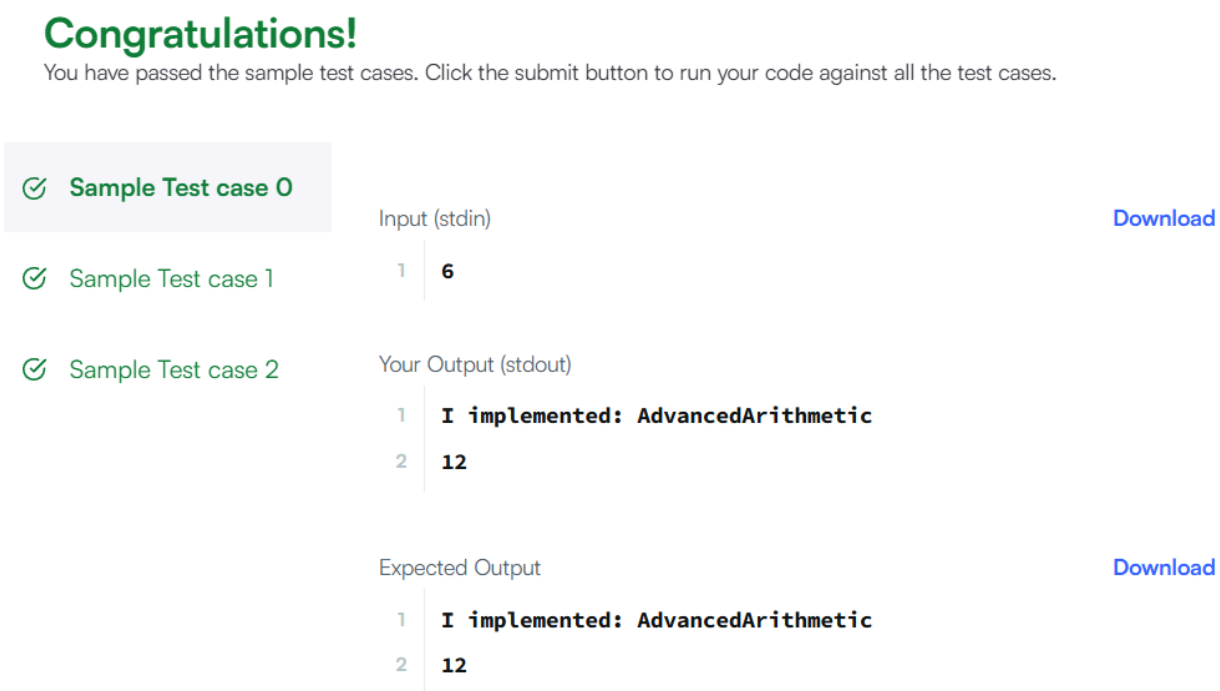
Output :



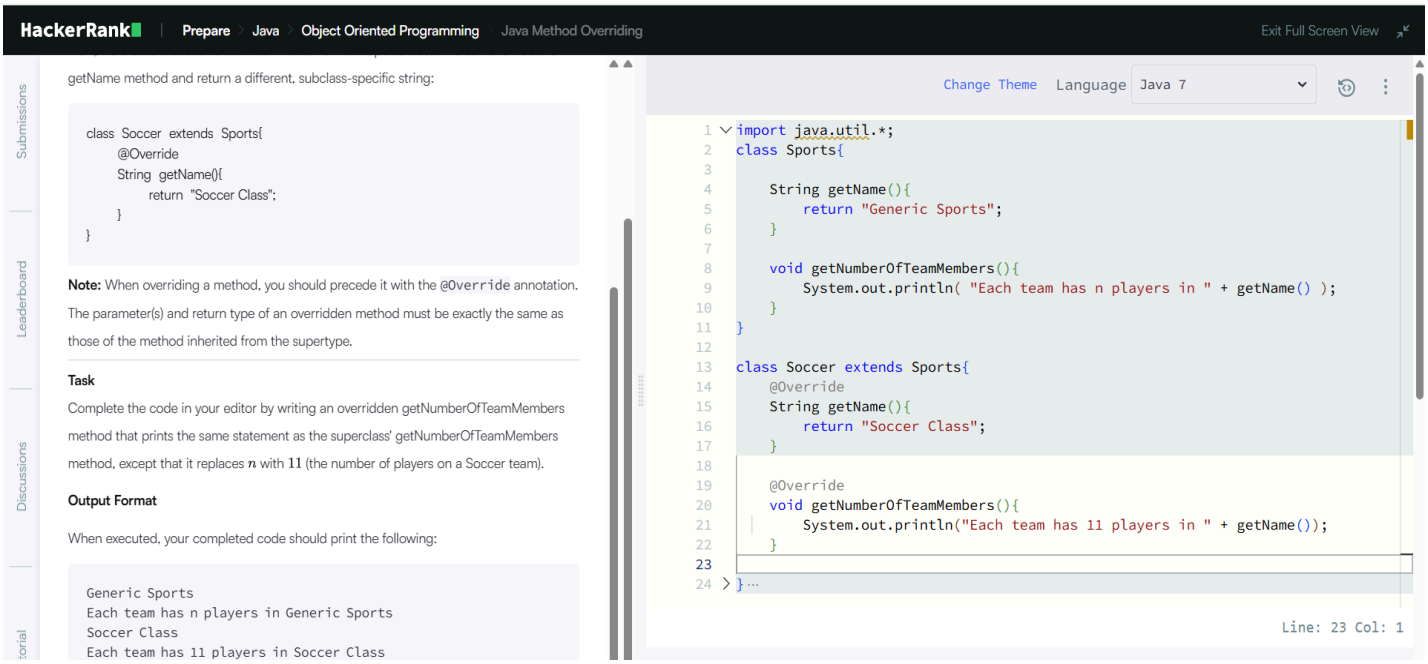
45 . Code :



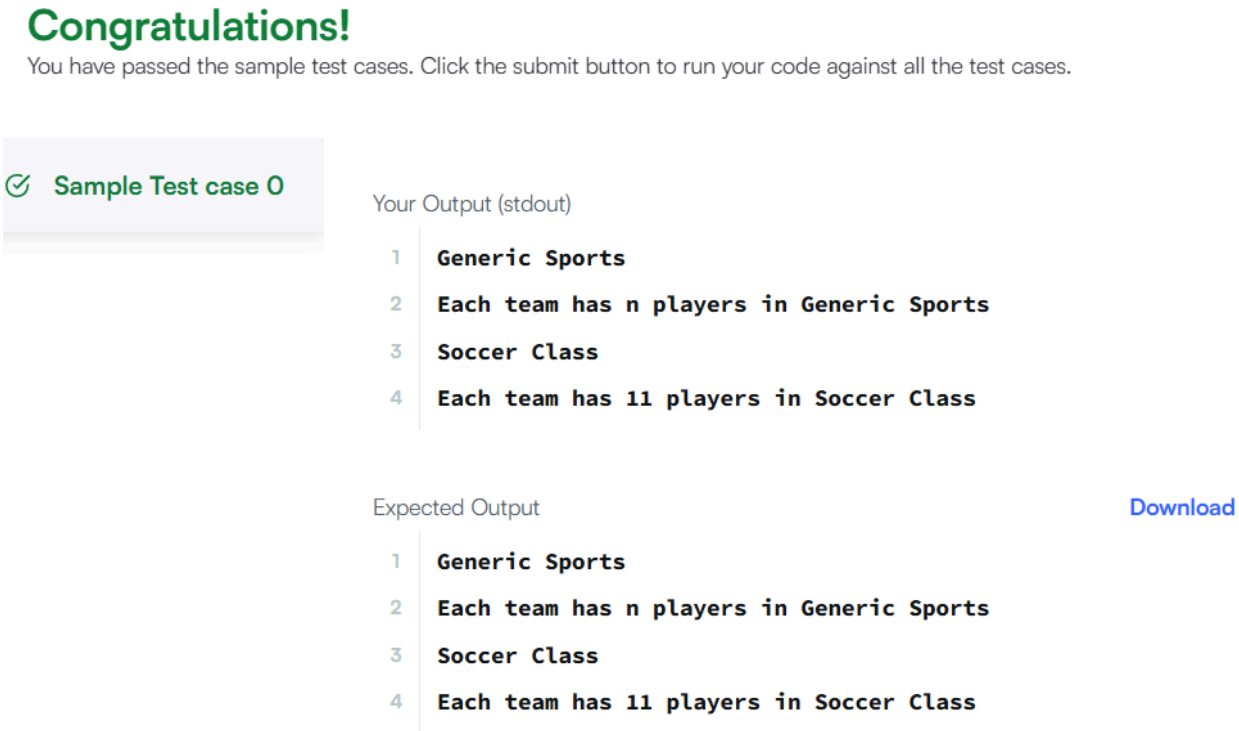
Output :



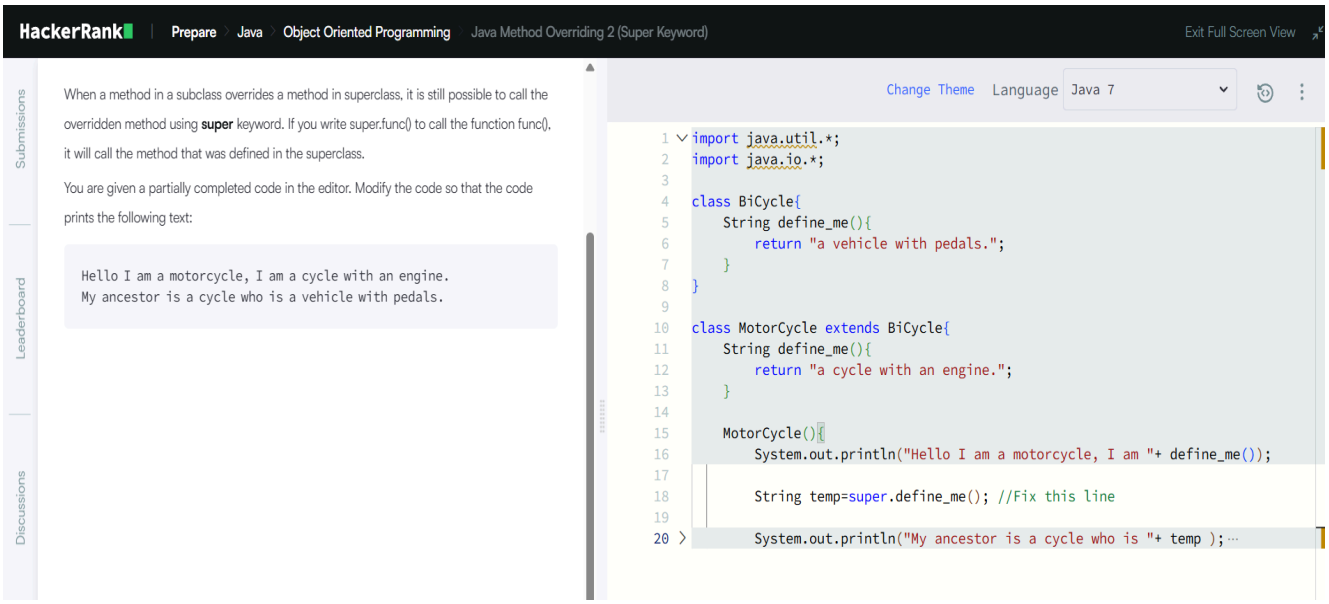
46. Code :



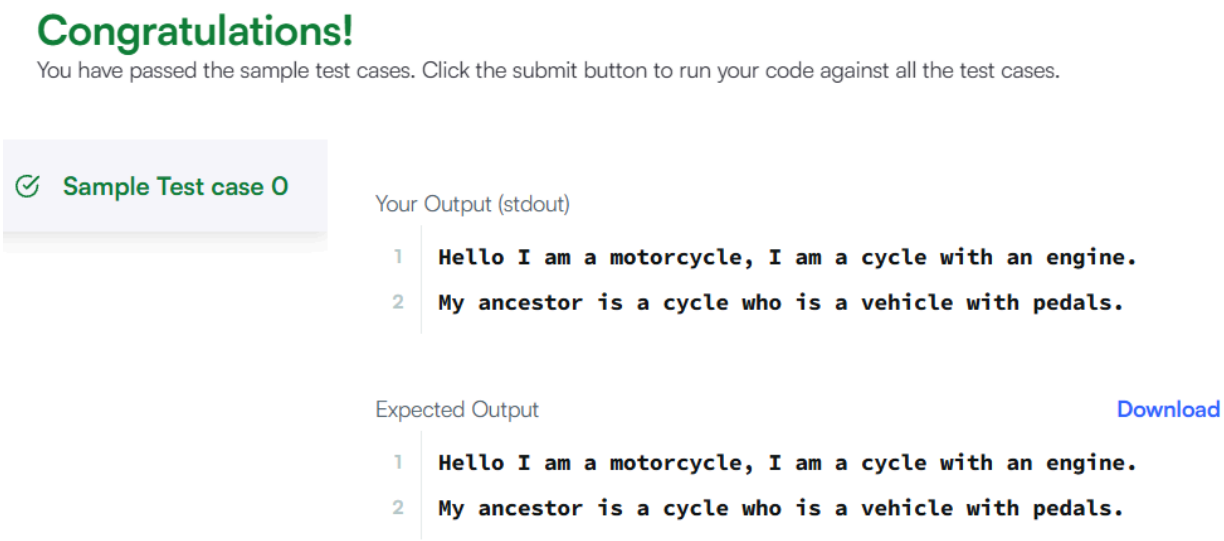
Output :



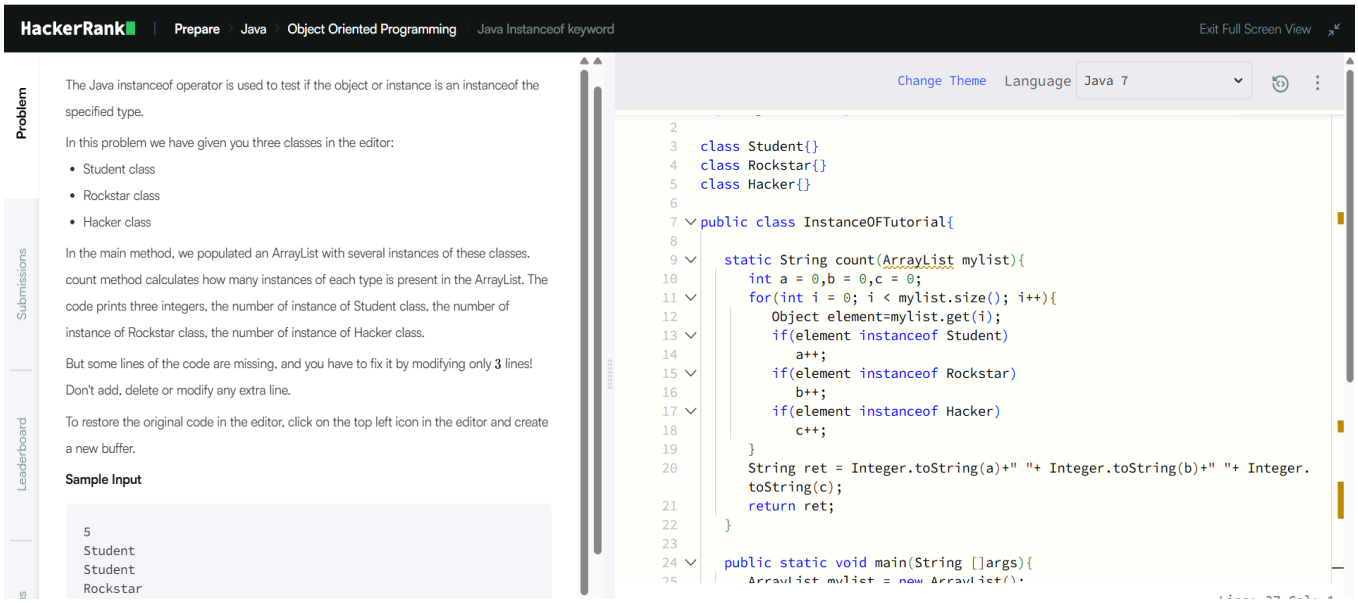
47 . Code :



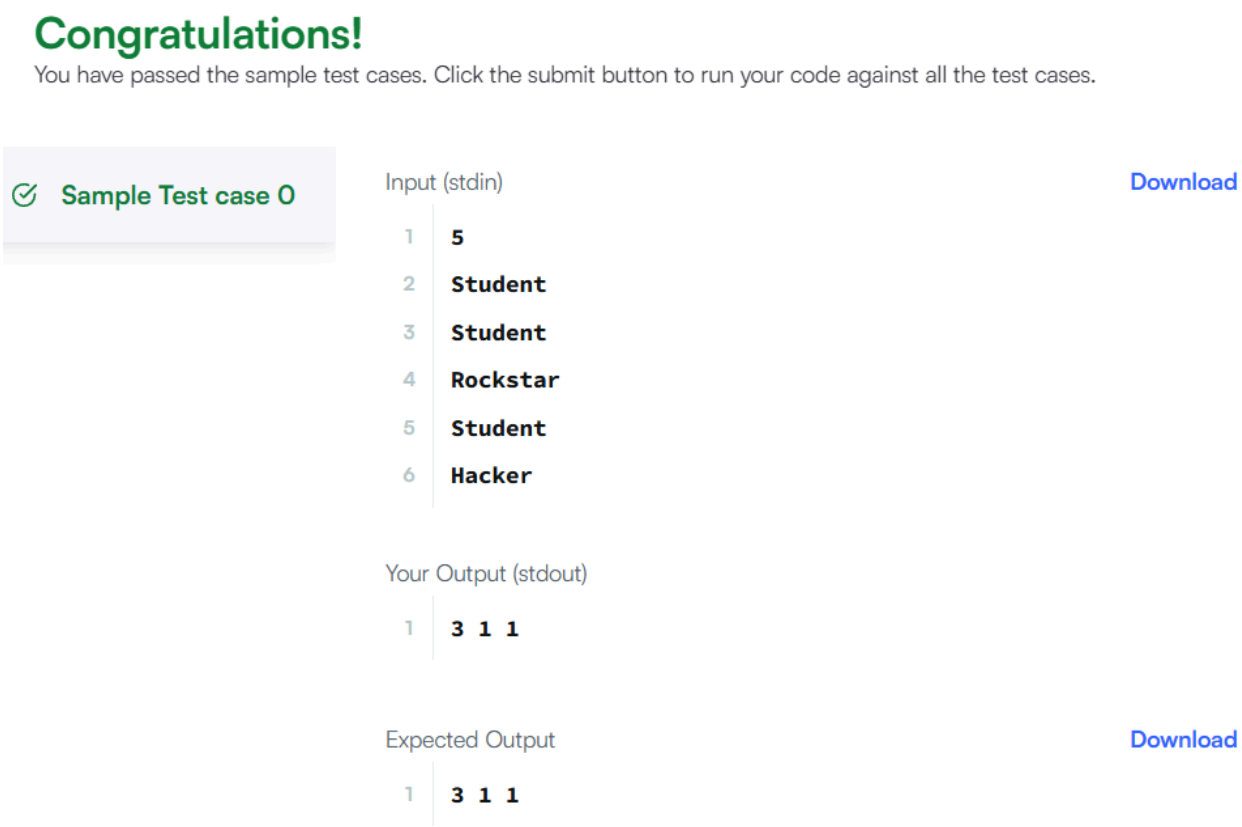
Output :



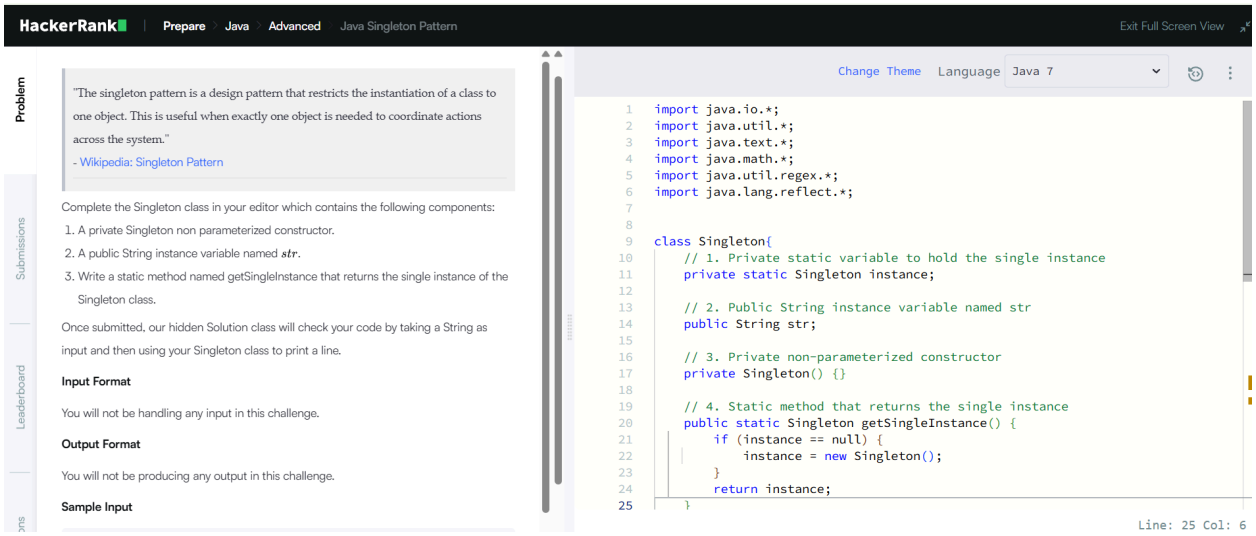
48 . Code :



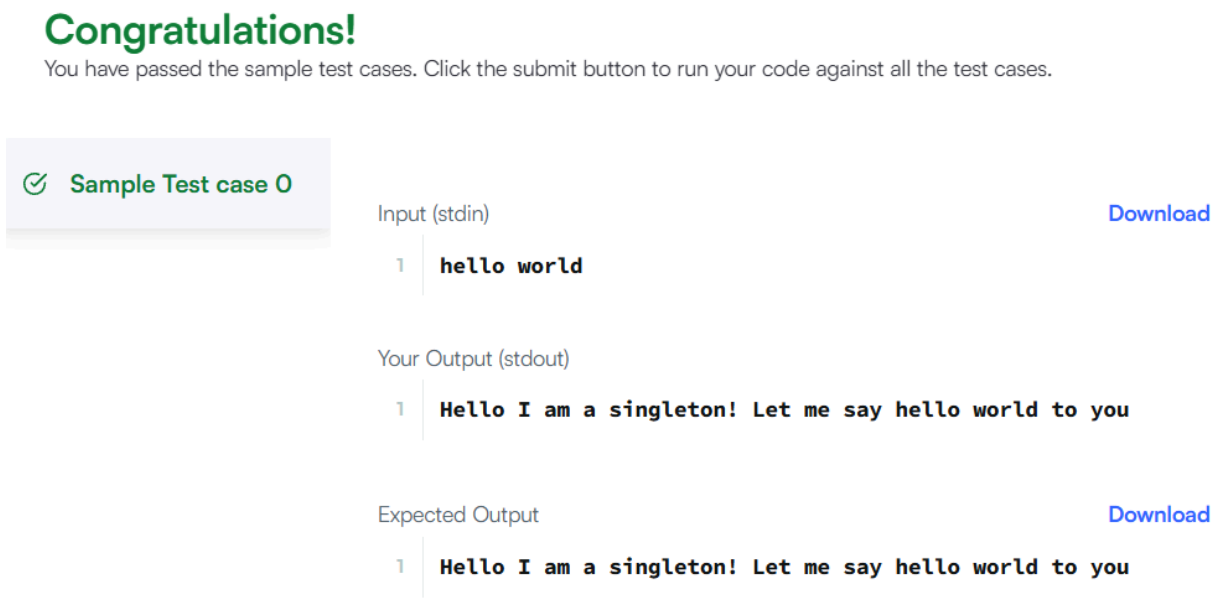
Output :



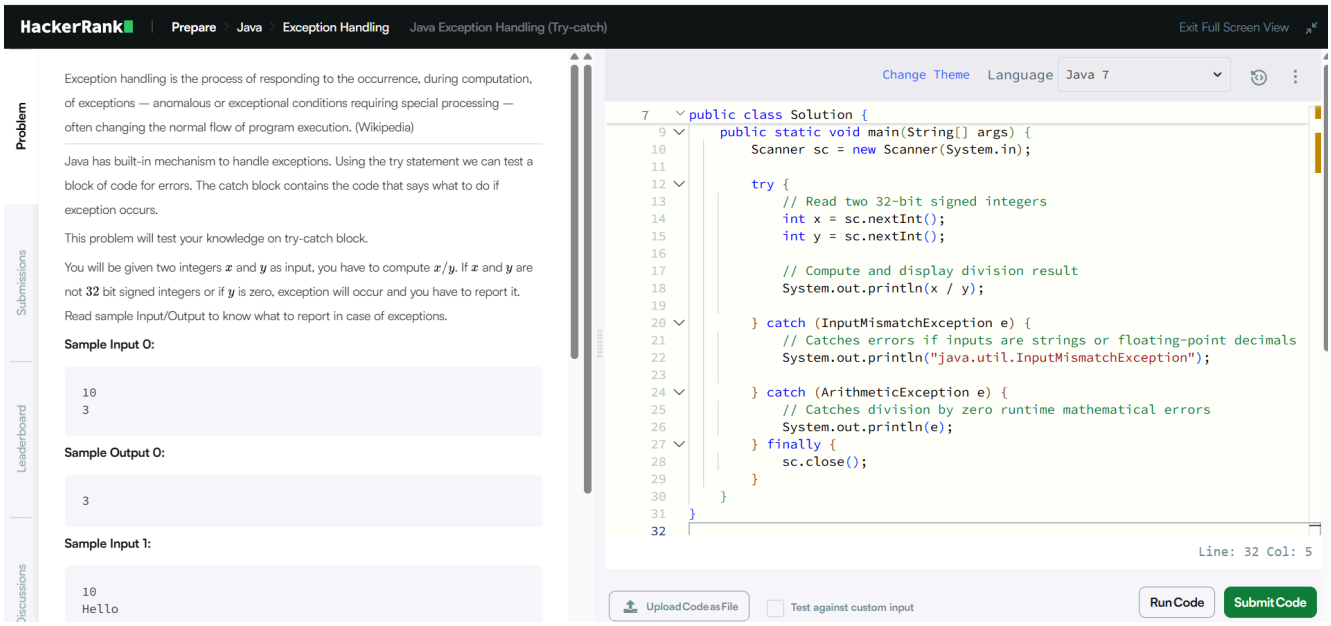
49 . Code :



Output :



50 . Code :



Output :